

Documentación académica y técnica

Requerimientos, diseño, operación y validación

Versión documental 1.2 · 17 de septiembre de 2026

Expediente para evaluación académica y mantenimiento del sistema. Diferencia capacidades observadas, requisitos propuestos y pruebas pendientes. Autor, institución, programa, docente y aprobación: por completar.

Contenido

- 01. Presentación y alcance
 - 02. Documento de requerimientos
 - 03. Casos de uso y procesos
 - 04. Arquitectura y diseño técnico
 - 05. Modelo entidad-relación
 - 06. Diccionario de datos
 - 07. Referencia de API
 - 08. Instalación, despliegue y operación
 - 09. Manual de usuario
 - 10. Plan de pruebas y matriz de trazabilidad
 - 11. Brechas, riesgos y evolución
- Tienda por empresa, domicilios y fidelización configurable
- Catálogo de rutas declarado en código

01. Presentación y alcance

Identificación

Nombre: PowerPOS Pioneers. **Tipo:** aplicación web de punto de venta y gestión comercial con orientación SaaS multiempresa. **Usuarios previstos:** restaurantes, bares, supermercados y otros comercios con requerimientos POS. Esta caracterización procede del README y los módulos implementados; no es el resultado de un estudio de mercado.

Problema y justificación

La operación gastronómica necesita mantener relacionados los pedidos, sus modificaciones, la preparación en cocina, las existencias de ingredientes y los ingresos del negocio. Llevar estos procesos en registros separados dificulta rastrear qué se vendió, qué se consumió y quién intervino. PowerPOS centraliza esas actividades y ofrece pantallas según el trabajo de caja, cocina y administración.

Se espera reducir errores de transcripción y facilitar el seguimiento operativo. Estos beneficios son objetivos del proyecto; no se dispone de mediciones que permitan cuantificar una mejora frente al proceso anterior.

Objetivo general

Desarrollar y mantener una plataforma web que integre ventas, preparación, caja, inventario y seguimiento administrativo de empresas, con usuarios autenticados y separación de información por empresa.

Objetivos específicos

1. Registrar pedidos con cantidades, exclusiones de ingredientes, adicionales y método de pago.
2. Comunicar pedidos a cocina y registrar su estado de preparación y entrega.
3. Relacionar ventas con caja, cliente, vendedor y movimientos financieros.
4. Administrar productos, categorías, recetas, ingredientes y preparaciones por lotes.
5. Ofrecer reportes operativos y seguimiento de consumo interno.
6. Gestionar empresas, sucursales, usuarios y capacidades habilitadas.
7. Establecer documentación, criterios de aceptación y una ruta de mejora verificable.

Alcance observado

Tienda web automática por empresa, catálogo público, solicitudes de domicilio, bandeja de cajero, asignación y seguimiento de entregas; fidelización configurable por empresa. Autenticación JWT; catálogo y categorías jerárquicas; recetas de ingredientes; adicionales; pedidos; cocina; caja y eventos; inventario y ajustes; preparaciones y lotes; clientes y puntos; ingresos y egresos; reportes; impresión ESC/POS; notificaciones; configuración; superadministración; auditoría de determinadas operaciones; consumo de empleados; pantalla de cliente y llamado.

La separación multiempresa es **parcial**: ingredientes y movimientos de inventario no tienen un propietario empresarial directo; algunos accesos por identificador tampoco filtran empresa. La existencia de planes y permisos no demuestra restricciones completas en el servidor.

Fuera del alcance implementado identificado

- Emisión completa de factura electrónica: se encontró una bandera de configuración, no un módulo de emisión.
- Procesamiento bancario de pagos: los métodos de pago se registran, sin pasarela identificada.
- Contabilidad de partida doble, liquidación de nómina o cuentas por pagar. **NOMINA** es una categoría de egreso.
- Operación sin conexión y sincronización posterior.
- Optimización de rutas y geocodificación: hay asignación/seguimiento de domicilios, sin cálculo de rutas o distancias.
- Control de vencimientos, asignación de ventas a lotes y descuento de porciones preparadas.
- Infraestructura productiva desplegada, disponibilidad garantizada o recuperación comprobada.

Actores y responsabilidades previstas

Actor	Responsabilidad de negocio
SUPERADMIN	Administrar empresas, planes, capacidades y consulta global de auditoría
ADMIN_EMPRESA	Configurar el negocio y supervisar su operación
GERENTE	Supervisar ventas, caja, inventario y reportes
CAJERO	Registrar pedidos y pagos de la caja asignada
COCINERO	Consultar pedidos y actualizar preparación
DOMICILIARIO	Consultar pedidos y actualizar estado según las rutas autorizadas
Cliente final	Recibir el pedido y consultar la pantalla de atención; no es una cuenta Usuario
Servicios externos	SMTP, Twilio e impresora; intervienen cuando están configurados

Esta tabla define responsabilidades previstas. Los permisos reales de API están en el documento 07; no todos los controladores restringen por rol.

Supuestos y restricciones

El navegador accede a una API central y esta a PostgreSQL. El entorno local usa API en 3000 y frontend en 3001. El cliente HTTP principal y la tienda usan **NEXT_PUBLIC_API_URL** ; quedan referencias locales de SSE/impresión por revisar antes del despliegue. El sistema usa nombres y formato monetario colombiano, pero no hay un campo de moneda por transacción: la moneda operativa debe acordarse con el negocio. Las horas del servidor influyen en cierres y reportes.

Glosario

Término	Significado
POS	Punto de venta para registrar pedidos y pagos
KDS	Pantalla de cocina para seguir la preparación
Comanda	Instrucción de preparación enviada a cocina, usualmente impresa
Receta	Ingredientes y cantidades asociados a un producto o preparación
Adicional	Extra con precio y, opcionalmente, consumo de un ingrediente
Exclusión	Nombre de ingrediente que no se utiliza en un producto vendido
Lote	Registro de producción de una preparación y sus insumos consumidos
Caja	Sesión de operación con base inicial, responsable y cierre
SSE	Canal HTTP de eventos enviados del servidor al navegador
FK / PK	Clave foránea / clave primaria
Auditoría	Registro de determinadas acciones, actor y entidad afectada

Fuentes

README original, módulos del backend, esquema de datos y navegación.

Tienda, entrega y presencia comercial

Cada empresa obtiene una página en `/tienda/{slug}` con su marca y productos. Su catálogo se mantiene desde el POS. La landing de `landing/` promociona PowerPOS; las tiendas se sirven desde la aplicación Next.js y requieren el despliegue de API y web. Se ofrece a restaurantes, bares, supermercados y comercios, con alcance específico verificado durante la demo.

La empresa define cuánto comprar para ganar un punto y cuánto descuento representa al canjearlo. El programa inicia desactivado, conserva los saldos anteriores y permite excluir categorías y productos. La base elegible recibe su parte proporcional de los descuentos; domicilio excluido y redondeo hacia abajo. No hay valor obligatorio.

02. Documento de requerimientos

Propósito y convención

Especificar qué debe ofrecer PowerPOS y contrastarlo con la implementación. Prioridades: **A** alta, **M** media. Estados: observado, parcial o propuesto; ninguno equivale a una prueba aprobada. Los criterios de aceptación definen resultados a comprobar. Toda aprobación de alcance está pendiente del responsable del proyecto.

Requerimientos funcionales

ID	Requerimiento y criterio de aceptación	Prioridad	Estado y evidencia
RF-01	Autenticar con correo y contraseña. Con datos válidos de usuario y empresa activos, devolver JWT y perfil; rechazar credenciales incorrectas.	A	Observado: <code>auth/auth.service.ts</code>
RF-02	Registrar empresa, sucursal principal y administrador; rechazar NIT o correo ya registrados.	A	Observado; creación de administrador separada de empresa: <code>auth/auth.service.ts</code>
RF-03	Administrar sucursales y su estado dentro de la empresa. Una consulta de listado no debe devolver sucursales ajenas.	A	Observado: <code>sucursales</code>
RF-04	Crear y gestionar usuarios, perfil, estado y permisos. El acceso debe respetar el rol y empresa.	A	Parcial: <code>usuarios</code> , controles de rol no uniformes
RF-05	Administrar categorías y subcategorías con nombre, presentación y estado.	M	Observado: <code>categorias</code> , relación <code>CategoriaPadre</code>
RF-06	Administrar productos con precio, categoría, disponibilidad y receta. No vender productos inactivos o no disponibles.	A	Observado: <code>productos</code> , <code>pedidos</code>
RF-07	Configurar adicionales y cobrarlos por unidad de producto; rechazar adicionales no permitidos. Conservar nombre y precio de la venta.	A	Observado: <code>adicionales</code> , <code>DetallePedidoAdicional</code>
RF-08	Registrar pedido con uno o más productos, cantidades positivas, cliente opcional, descuento y pago. Recalcular importes en servidor.	A	Parcial: cálculo observado; validación integral de cantidades y descuento pendiente
RF-09	Exigir caja abierta en la sucursal y vendedor asignado o supervisor para registrar ventas.	A	Observado: <code>pedidos.crearPedido</code>
RF-10	Descontar ingredientes de receta respetando exclusiones y consumo de adicionales.	A	Observado; atomicidad y control de stock insuficientes: <code>pedidos</code>
RF-11	Consultar pedidos y actualizar estados de preparación y entrega dentro de la empresa.	A	Parcial: actualización observada; transiciones válidas no controladas
RF-12	Actualizar cocina cuando se crea o modifica un pedido. Aceptar eventos SSE y recuperación por consulta periódica.	A	Observado: <code>pedidos-eventos</code> , <code>web/app/cocina</code>
RF-13	Abrir caja con base y responsable; impedir segunda caja abierta por sucursal. Cerrar mostrando ventas, esperado y diferencia.	A	Observado por lógica de servicio; concurrencia y autorización pendientes

ID	Requerimiento y criterio de aceptación	Prioridad	Estado y evidencia
RF-14	Registrar eventos de apertura, cierre e irregularidad; generar alerta por diferencia absoluta superior a 1000.	M	Observado: caja
RF-15	Gestionar ingredientes, unidad de compra, conversión, costo y stock mínimo; registrar entradas, salidas y ajustes con historial.	A	Observado; inventario global sin aislamiento empresarial
RF-16	Crear preparaciones y lotes con insumos usados y cantidades producidas; disminuir stock y registrar historial de producción.	A	Observado: preparaciones
RF-17	Asociar preparaciones a productos y conocer saldo real de porciones tras ventas.	M	Parcial: asociación existe; disponible suma producción sin restar consumos
RF-18	Administrar clientes y saldos por empresa; acumular y canjear puntos con reglas configurables y snapshot por venta.	A	Implementado: clientes, pedidos, tienda; HTTP probado
RF-19	Registrar ingreso de venta y movimientos manuales de ingreso o egreso; filtrar y resumir por empresa y período.	A	Observado: financiero , pedidos
RF-20	Consultar estadísticas de ventas y reportes de períodos, empleados, clientes y rentabilidad.	M	Observado: reportes , pedidos.obtenerEstadísticas
RF-21	Imprimir comanda o recibo y accionar cajón cuando el hardware esté configurado. Informar impresión no realizada.	M	Observado: impresion ; requiere prueba con equipo real
RF-22	Mostrar resumen y llamado al cliente con comunicación entre ventanas del mismo origen.	M	Observado: /cliente , /llamado ; no sincroniza equipos por sí solo
RF-23	Enviar alertas y felicitaciones mediante los canales configurados; funcionar con integraciones opcionales deshabilitadas.	M	Observado: notificaciones , tareas ; entrega externa no validada
RF-24	Permitir al SUPERADMIN gestionar estado, plan, permisos y capacidades de las empresas; consultar auditoría.	A	Observado: superadmin ; enforcement global de planes es parcial
RF-25	Registrar consumo de empleados solo para ADMIN_EMPRESA o GERENTE y empresa habilitada; descontar receta sin crear venta.	A	Observado: consumo-empleados
RF-26	Registrar actor, empresa y detalle de operaciones auditadas.	M	Parcial: hay auditoría en operaciones seleccionadas, no cobertura total
RF-27	Anular una venta restaurando consistentemente inventario, puntos y efecto financiero según política aprobada.	A	Pendiente: restricciones protectoras implementadas, reversión integral no disponible
RF-28	Emitir factura electrónica y conservar respuesta del proveedor, estados y documento.	M	Propuesto: solo existe bandera de habilitación

Las rutas de evidencia son relativas a `api/src` salvo indicación contraria. Para pruebas relacionadas, ver trazabilidad.

Requerimientos no funcionales

Metas propuestas para acordar, no resultados medidos ni compromisos de servicio existentes.

ID	Requerimiento	Criterio verificable propuesto	Estado
RN F-01	Aislamiento multiempresa	Ninguna consulta o modificación de A debe alcanzar recursos de B, incluyendo inventario, clientes y caja. Probar matriz de IDs cruzados.	Parcial
RN F-02	Autorización de servidor	Toda acción sensible exige rol/capacidad autorizados aunque se invoque fuera de la interfaz.	Parcial
RN F-03	Integridad transaccional	Venta, stock, ingreso y puntos se confirman o revierten juntos ante fallo inducido.	Implementado en ventas; pruebas HTTP de fallo/concurrencia, cobertura de fallos por ampliar
RN F-04	Validación de entradas	Rechazar cantidades no positivas, descuentos fuera de rango, valores no finitos, estados inválidos e IDs ajenos con error controlado.	Parcial
RN F-05	Protección de credenciales	Contraseñas almacenadas como hash; secreto JWT externo obligatorio; HTTPS en despliegue; tokens excluidos de logs.	bcrypt observado; resto por verificar/implementar
RN F-06	Rendimiento	Meta inicial: percentil 95 menor a 2 s para consulta y registro de pedido con 20 usuarios concurrentes y 10000 pedidos; excluir impresión externa, medir aparte notificaciones.	Propuesto, ambiente por acordar
RN F-07	Continuidad	Meta inicial: respaldo diario, pérdida máxima de 24 h y recuperación en 4 h; validar restauración periódica en ambiente aislado.	Propuesto
RN F-08	Usabilidad	Completar una venta con teclado y pantalla táctil; controles legibles, errores visibles y confirmación de resultado sin duplicar operación.	Interfaz observada; evaluación pendiente
RN F-09	Compatibilidad	Ejecutar los flujos principales en navegadores de escritorio seleccionados por el equipo; verificar SSE, BroadcastChannel y voz.	Pendiente de matriz de versiones
RN F-10	Mantenibilidad	Cambios de esquema incluyen migración y actualización de documentación; módulos con responsabilidades delimitadas.	Estructura observada; proceso propuesto
RN F-11	Observabilidad	Registrar errores con correlación, sin contraseña/token; medir fallos de venta e integraciones.	Propuesto
RN F-12	Escalabilidad	Antes de usar varias instancias, distribuir eventos y evitar tareas programadas duplicadas.	Pendiente; eventos hoy en memoria

Reglas de negocio observadas

ID	Regla
RN -01	El NIT de empresa, correo de usuario y número de pedido son únicos globalmente. Documento de cliente es único junto con empresa cuando tiene valor.

ID	Regla
RN-02	Roles: SUPERADMIN, ADMIN_EMPRESA, GERENTE, CAJERO, COCINERO y DOMICILIARIO.
RN-03	Estados de pedido: PENDIENTE, EN_COCINA, LISTO, ENTREGADO y ANULADO. El enum no impone secuencia.
RN-04	Pagos registrados: EFECTIVO, TARJETA, TRANSFERENCIA, NEQUI y DAVIPLATA. No se verifica pago ante un proveedor.
RN-05	Subtotal de línea = (precio base + suma de precio adicional × cantidad adicional) × cantidad del producto. Total = suma de líneas – descuento manual – descuento de canje + costo de domicilio.
RN-06	Exclusiones se comparan con el nombre del ingrediente; renombrar ingredientes puede afectar esa interpretación.
RN-07	Si el producto tiene adicionales asociados, se toman los activos/disponibles de esa lista. Si no tiene, admite el catálogo activo/disponible de empresa cuando aceptaAdicionales es true.
RN-08	Base de caja no es venta. El cierre actual suma ventas no anuladas de todos los métodos de pago a la base; no es un arqueo exclusivo de efectivo.
RN-09	Un cron revisa cajas cada cinco minutos. Horario codificado: lunes a viernes 15:00–22:00; sábado y domingo 12:00–22:00, usando hora local del servidor. Fuera del horario cierra con monto esperado.
RN-10	Stock bajo: stock ≤ mínimo. Stock crítico: stock ≤ 50 % del mínimo.
RN-11	La empresa define cuánto comprar para ganar un punto y cuánto descuento representa al canjearlo. El programa inicia desactivado, conserva los saldos anteriores y permite excluir categorías y productos. La base elegible recibe su parte proporcional de los descuentos; domicilio excluido y redondeo hacia abajo. No hay valor obligatorio.
RN-12	Consumo de empleado guarda nombre libre del consumidor y usuario que registra; no tiene FK a un empleado consumidor. El valor estimado usa precio de venta de referencia, no costo de ingredientes.
RN-13	Planes configurables: BASICO, MEDIUM y PREMIUM. Modos de preparación: KDS y COMANDAS.

Decisiones pendientes del negocio

Confirmar moneda y zona horaria, reglas de descuento, política de stock negativo, anulación/devolución, arqueo por método de pago, horarios por sucursal, valores de fidelización elegidos por cada empresa, matriz definitiva de permisos y requisitos de facturación. Estas decisiones afectan la aceptación; no deben deducirse de etiquetas de interfaz.

Requerimientos integrados en 1.2

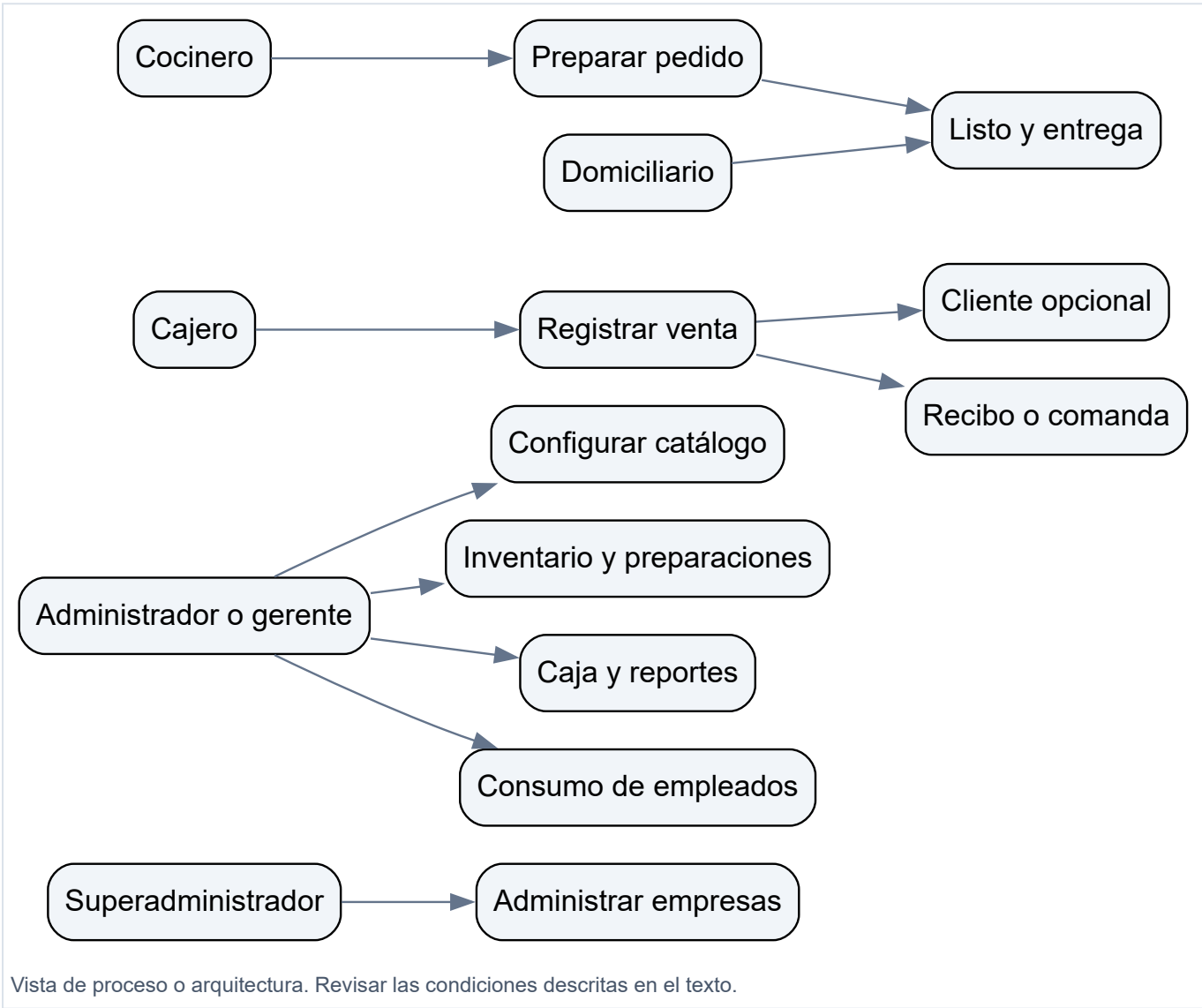
ID	Requerimiento	Criterio de aceptación	Estado
RF-29	Crear tienda automáticamente para cada empresa y permitir personalizar enlace, marca, contenido y visibilidad.	Enlace único; pública solo con empresa activa y tienda publicada; configuración reservada a administrador.	Implementado
RF-30	Mostrar catálogo público conectado a productos, precios, categorías y disponibilidad del POS.	Solo catálogo de esa empresa; sin correos internos, usuarios ni saldos de clientes.	Implementado

ID	Requerimiento	Criterio de aceptación	Estado
RF-31	Recibir solicitudes web con zona, dirección, teléfono, carrito y forma de pago.	Precios y tarifa recalculados por servidor; reintento con misma clave devuelve la misma solicitud.	Implementado
RF-32	Permitir revisar, aceptar o rechazar solicitudes en la bandeja del cajero.	Aviso cada 5 s; caja y sucursal válidas; aceptación atómica y única; rechazo con motivo sin venta.	Implementado
RF-33	Asignar y seguir domicilios hasta su entrega.	Domiciliario de la empresa ve sus asignaciones; entrega actualiza PedidoWeb y Pedido.	Implementado
RF-34	Ofrecer pedido por WhatsApp de cada empresa.	Abre conversación con carrito; no envía ni importa mensajes automáticamente.	Implementado
RF-35	Configurar acumulación y canje por empresa, excluyendo productos/categorías.	Inactivo por defecto; valores positivos al activar; domicilio excluido; canje no supera saldo ni productos.	Implementado
RF-36	Proteger recepción pública y seguimiento.	Límites iniciales de solicitudes, identificador no secuencial, seguimiento sin dirección ni teléfono; sin consulta/canje público de puntos.	Implementado; endurecimiento distribuido pendiente

03. Casos de uso y procesos

Los siguientes flujos describen el uso previsto y señalan excepciones de la implementación. Actor autorizado significa autorización que debe comprobarse en servidor; la visibilidad del menú no basta.

Mapa de casos de uso



Mapa funcional representado como diagrama de flujo; no pretende ser notación UML estricta.

CU-01. Iniciar sesión

Actor: usuario registrado. **Requisito:** RF-01. **Precondición:** API y base disponibles, usuario activo.

1. Abrir `/login` e ingresar correo y contraseña.

2. El servidor busca usuario y verifica su estado, el de empresa y el hash.
3. Devuelve token de ocho horas y perfil.
4. La aplicación guarda sesión y dirige a la pantalla correspondiente.

Alternativa: datos incorrectos o empresa inactiva producen rechazo; SUPERADMIN tiene excepción al bloqueo empresarial en login. **Resultado:** sesión iniciada. **Límite:** la estrategia JWT reconstruye identidad desde token y no vuelve a consultar actividad en cada petición.

CU-02. Preparar catálogo

Actor previsto: administrador. **Requisitos:** RF-05 a RF-07, RF-15. **Precondición:** empresa y categorías creadas.

1. Crear ingredientes con unidades y stock.
2. Crear categoría o subcategoría.
3. Crear producto y definir precio, disponibilidad e ingredientes con cantidades.
4. Configurar adicionales y su consumo opcional de ingrediente.
5. Consultar el producto en POS.

Alternativa: producto no disponible no debe venderse. **Resultado:** catálogo operativo. **Límite:** los ingredientes son globales en el modelo actual; no tratarlos como inventario aislado de cada empresa.

CU-03. Abrir caja

Actor previsto: administrador o gerente. **Requisitos:** RF-09, RF-13, RF-14. **Precondición:** sucursal y usuario responsable activos.

1. Abrir Dashboard y seleccionar apertura.
2. Indicar monto base y cajero.
3. El servidor valida base no negativa y ausencia de caja abierta.
4. Crea caja, evento y solicita notificación.

Alternativas: caja ya abierta o cajero inexistente: rechazo. **Resultado:** caja disponible para ventas. **Límite:** la restricción de actor no está declarada mediante RolesGuard en este controlador; existe brecha de autorización.

CU-04. Registrar venta

Actor: cajero asignado o supervisor. **Requisitos:** RF-07 a RF-10, RF-18, RF-19. **Precondición:** caja abierta, productos activos y sucursal de la empresa.

1. Elegir productos, cantidades, exclusiones y adicionales.
2. Asociar cliente si corresponde y elegir método de pago.
3. Enviar pedido.
4. El servidor valida caja, sucursal, cliente, producto y adicionales; calcula importes.
5. Guarda pedido y detalles, descuenta stock, crea ingreso y suma puntos.
6. Emite evento, solicita notificación y devuelve pedido.
7. La interfaz gestiona recibo, comanda y cajón según configuración.

Alternativas: caja cerrada, vendedor no autorizado, producto indisponible o adicional ajeno: rechazo. **Resultado normal:** venta identificada por número. **Integridad:** pedido, stock, ingreso, canje y acumulación se guardan en una transacción. Si se pierde la respuesta de una venta POS, consultar antes de repetirla: su endpoint aún no tiene

clave de idempotencia.

CU-05. Preparar y entregar

Actor: cocinero o personal autorizado. **Requisitos:** RF-11, RF-12, RF-22. **Precondición:** pedido registrado.

1. Consultar cocina y revisar cantidades, exclusiones y extras.
2. Marcar preparación, luego listo.
3. Activar llamado al cliente cuando corresponda.
4. Entregar y registrar ENTREGADO.

Alternativa: si falla SSE, la cocina vuelve a consultar cada cinco segundos. **Resultado:** estado actualizado.

Límites: transiciones no validadas como máquina de estados; pantallas de llamado usan comunicación del navegador, no un canal entre equipos.

CU-06. Ajustar inventario

Actor previsto: administrador o gerente. **Requisito:** RF-15. **Precondición:** ingrediente registrado.

1. Consultar stock e historial.
2. Elegir ENTRADA, SALIDA o AJUSTE e indicar cantidad y descripción.
3. Indicar si la cantidad se expresa en unidad de compra.
4. El servicio aplica factor si corresponde; entrada suma, salida resta y ajuste sustituye saldo.
5. Guarda stock, movimiento y evalúa alerta.

Resultado: nuevo saldo e historial. **Límite:** no se garantiza rechazo de stock negativo ni transacción entre saldo y movimiento.

CU-07. Registrar lote

Actor previsto: responsable de inventario. **Requisitos:** RF-16, RF-17. **Precondición:** preparación existente.

1. Definir preparación y receta.
2. Registrar lote por API con producción y porciones positivas, fecha e insumos efectivamente usados.
3. El servidor crea lote y detalle; descuenta ingredientes y registra movimientos y auditoría.
4. Consultar lotes y porciones reportadas.

Resultado: producción registrada. **Límite:** API de lotes observada; no se identificó un formulario de captura de lotes en la pantalla de inventario revisada. Las porciones disponibles suman lo producido, sin descontar ventas.

CU-08. Cerrar caja

Actor previsto: administrador o gerente. **Requisitos:** RF-13, RF-14. **Precondición:** caja ABIERTA.

1. Revisar ventas y monto esperado en Dashboard.
2. Ingresar monto final y solicitar cierre.
3. El servidor calcula diferencia, cierra y registra evento.
4. Si diferencia absoluta supera 1000, registra alerta y solicita notificación.

Alternativa: cron fuera de horario cierra usando monto esperado. **Resultado:** caja CERRADA. **Límite:** monto esperado incorpora todos los métodos de pago; cierre automático no prueba conteo físico.

CU-09. Consumo de empleados

Actor: ADMIN_EMPRESA o GERENTE. **Requisito:** RF-25. **Precondición:** empresa con función habilitada.

1. Abrir Consumo staff, indicar nombre de empleado, productos y cantidades.
2. Registrar consumo.
3. El servidor guarda cabecera y líneas, descuenta receta y registra auditoría.
4. Consultar historial y estimado del mes.

Alternativa: empresa sin habilitación: 403; sin nombre o productos: 400. **Resultado:** consumo sin pedido de venta ni ingreso financiero. **Límite:** estimado usa precio de referencia, no costo real.

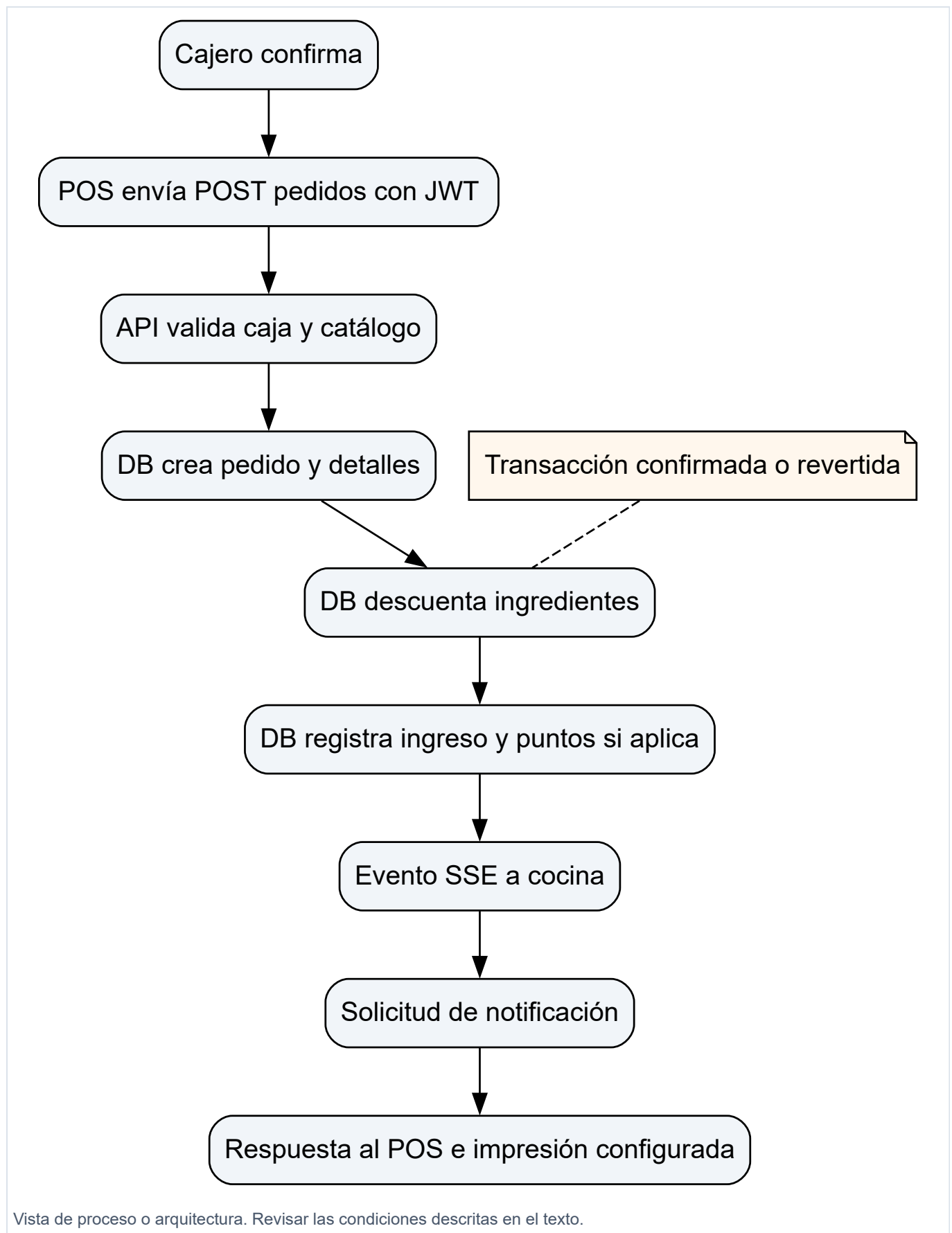
CU-10. Administrar empresa y entregar sistema

Actor: SUPERADMIN y responsable de implantación. **Requisitos:** RF-02, RF-24, RF-26.

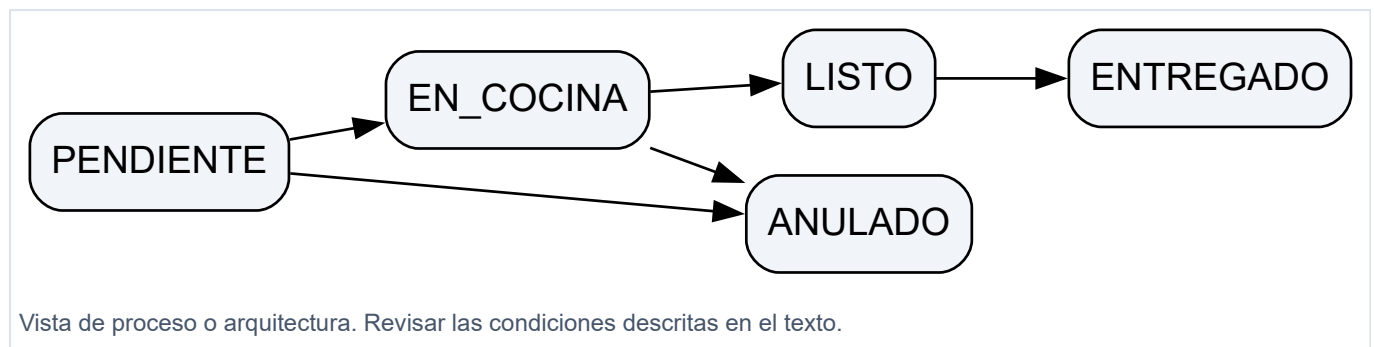
1. Crear empresa y administrador.
2. Configurar plan, modo de preparación y capacidades acordadas.
3. Configurar sucursal y personal; cargar catálogo revisado.
4. Ejecutar pruebas de aceptación y completar acta de entrega.
5. Entregar al adquirente URL, canales de soporte y manual de cliente.

Resultado: entrega documentada cuando las pruebas y condiciones se aprueben. Crear empresa o habilitar una bandera no equivale a desplegar ni a tener una integración externa operativa.

Secuencia actual de venta



Ciclo de estados esperado



Es un flujo de negocio propuesto para acordar. El código permite actualizar el enum sin comprobar el estado anterior; las condiciones de anulación y su reversión están pendientes.

CU-11. Administrar la tienda

Actor: ADMIN_EMPRESA. **Precondición:** empresa registrada. Abrir Mi tienda; personalizar enlace, título, descripción, portada, color y WhatsApp; mantener logo en Configuración y catálogo en Productos; definir sucursal, mínimo y zonas. Activar recepción solo después de revisar tarifas y operación. Rechazar enlace repetido o sucursal ajena. **Resultado:** página de empresa lista en su ruta del despliegue web.

CU-12. Comprar desde la web

Actor: comprador público. Consultar catálogo, agregar cantidades enteras, indicar datos de entrega y zona, revisar total y autorizar uso de datos. El servidor valida catálogo y precios. Crea RECIBIDO y devuelve referencia para seguimiento. **Alternativas:** tienda pausada, producto indisponible, zona no atendida o precio cambiado: rechazo controlado. Reintentar la misma clave no duplica la solicitud. No afecta caja, inventario ni puntos antes de revisión.

CU-13. Revisar y despachar un domicilio

Actores: cajero, administrador/gerente y domiciliario asignado. Revisar bandeja; contactar al comprador si es necesario y verificar medio de pago. Opcionalmente asociar cliente identificado. Aceptar con caja abierta convierte la solicitud en una venta única. Alternativa: rechazar RECIBIDO con motivo. Asignar repartidor activo de la misma empresa y marcar EN_CAMINO. Confirmar ENTREGADO actualiza la entrega y el pedido POS. Las solicitudes web RECIBIDO se pueden rechazar sin venta. Una venta web aceptada o con movimientos de puntos bloquea la anulación directa y requiere conciliación. No existe todavía un flujo automático de devolución que revierta ingreso e inventario; no presentar ANULADO como reembolso.

CU-14. Configurar y canjear fidelización

Actor configurador: ADMIN_EMPRESA. La empresa define cuánto comprar para ganar un punto y cuánto descuento representa al canjearlo. El programa inicia desactivado, conserva los saldos anteriores y permite excluir categorías y productos. La base elegible recibe su parte proporcional de los descuentos; domicilio excluido y

redondeo hacia abajo. No hay valor obligatorio. **Actor venta:** cajero autorizado. Seleccionar cliente identificado, elegir puntos a canjear y confirmar descuento. Servidor verifica y descuenta saldo en la misma transacción de venta; saldo insuficiente revierte la operación. En domicilio público el cajero vincula identidad al aceptar; no se confía en un teléfono público para acceder a puntos.

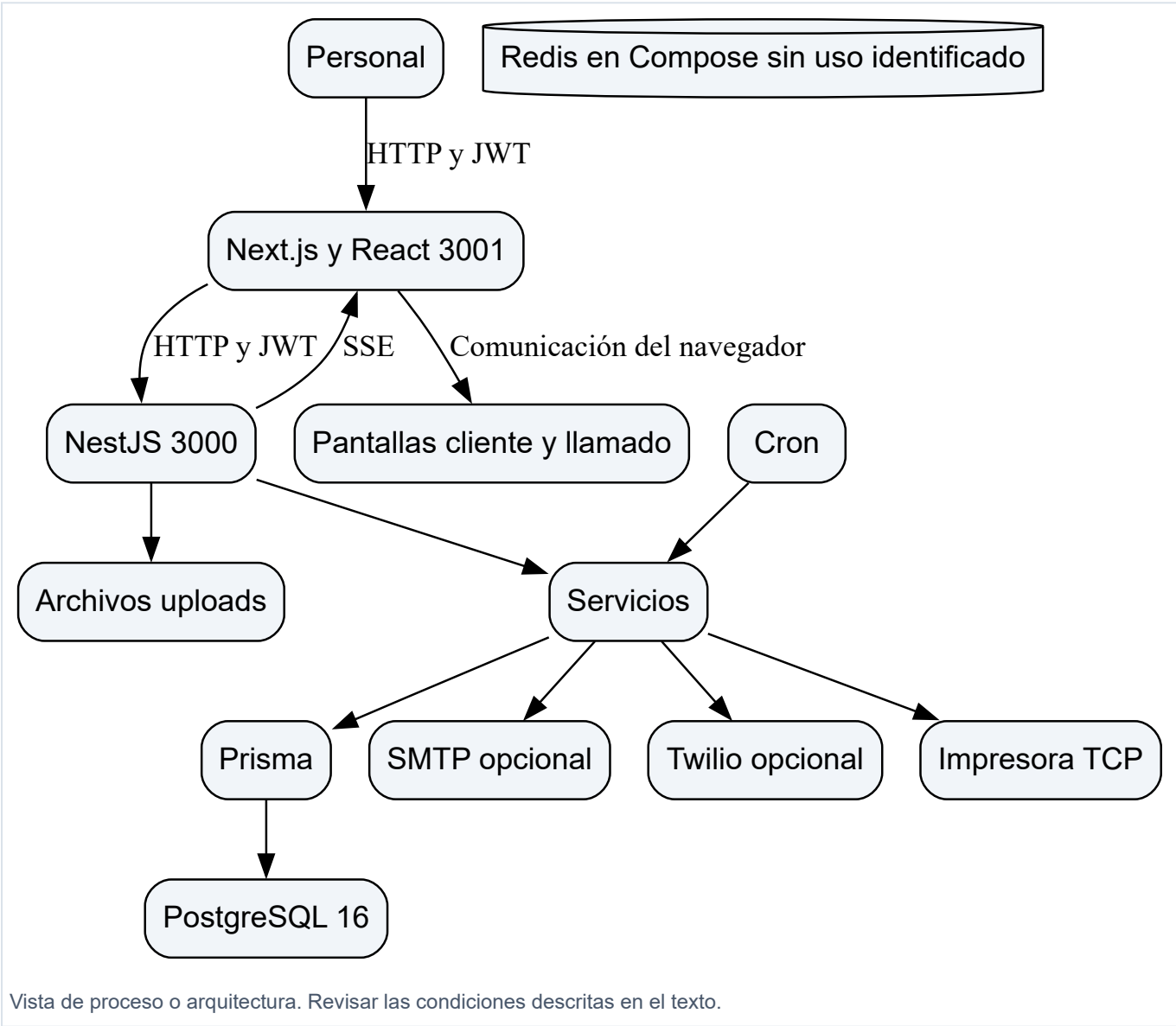
CU-15. Consultar por WhatsApp

Actor: comprador. Elige WhatsApp en la tienda; se abre conversación con el número configurado y texto del carrito. El comprador decide enviarlo. El comercio confirma disponibilidad y domicilio por ese canal y registra la venta por su procedimiento. La conversación no crea registros automáticos en PowerPOS.

04. Arquitectura y diseño técnico

Organización

PowerPOS tiene un frontend Next.js, una API NestJS organizada por módulos y persistencia PostgreSQL a través de Prisma. La API concentra lógica de negocio e integraciones. Es un backend modular único, no una arquitectura de microservicios.



Tecnologías declaradas

Capa	Tecnologías observadas
Interfaz	Next.js 16.2.9, React 19.2.4, TypeScript, Tailwind CSS 4

Capa	Tecnologías observadas
Estado y HTTP	Zustand 5, Axios
Visualización	Recharts, Lucide
API	NestJS 11, Express, TypeScript
Datos	PostgreSQL 16, Prisma 5.22
Identidad	Passport JWT, bcrypt; JWT con vigencia de 8 horas
Eventos	RxJS Subject y SSE; consultas periódicas de respaldo
Tareas	@nestjs/schedule
Integraciones	Nodemailer, Twilio, sockets TCP ESC/POS, Sharp
Infraestructura local	Docker Compose con PostgreSQL y Redis; aplicaciones Node separadas
Pruebas	Jest y Supertest en API; no script de pruebas del frontend identificado

Versiones tomadas de manifests del repositorio. No se afirma que sean las versiones más recientes ni se efectuó auditoría de dependencias.

Responsabilidades por componente

Componente	Responsabilidad
web/app	Páginas de operación y administración
web/components	Navegación y controles reutilizables, teclado táctil
web/lib/api.ts	Cliente Axios, incorporación del token y manejo de 401
web/store/authStore.ts	Sesión persistida en navegador y cierre al detectar cambio de día
Controladores NestJS	Rutas, extracción de parámetros y guards declarados
Servicios NestJS	Reglas operativas y acceso a datos
api/src/prisma	Cliente compartido de base de datos
api/prisma/migrations	Evolución del esquema
api/uploads	Logos publicados como /uploads/

Autenticación y autorización

Login compara bcrypt y emite JWT con `sub`, `email`, `rol`, `empresaId` y `sucursalId`. La estrategia acepta Bearer y también `token` en query, utilizado por SSE. El token se guarda en localStorage. Un 401 lleva al login.

RolesGuard interpreta decoradores Roles. Si no existe restricción de roles, permite continuar. Varios controladores solo usan JwtAuthGuard. Los permisos JSON y los filtros de navegación no equivalen a autorización de backend. La matriz real de decoradores se incluye en el catálogo de API.

Separación de empresas

Parte de las consultas usa `empresaId` del token, de forma directa o mediante sucursal. Las claves foráneas por sí solas no garantizan que dos entidades relacionadas pertenezcan a la misma empresa. Ingrediente carece de `empresaId`; clientes por ID y operaciones de caja requieren revisión adicional. Por ello no se declara aislamiento completo.

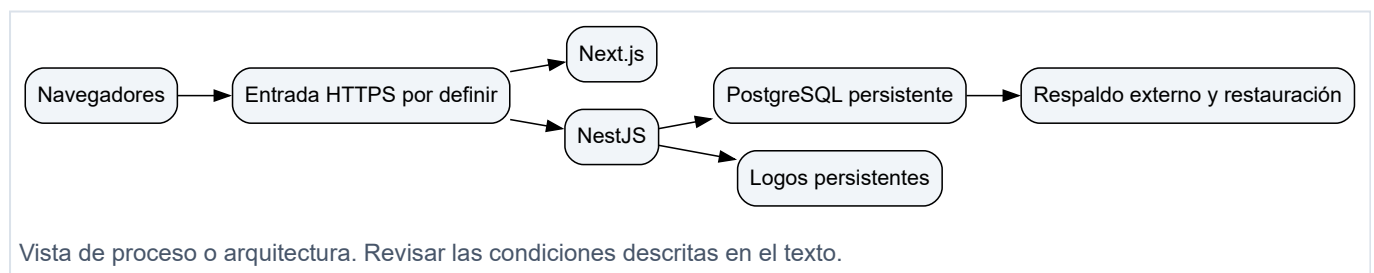
Actualización de pantallas

- Cocina usa EventSource hacia `/pedidos/stream` y consulta cada 5 segundos.
- Dashboard, clientes y financiero usan intervalos de 10 segundos; inventario, 15 segundos.
- Los eventos del backend se filtran por empresa y viven en la memoria del proceso; no se retienen para reproducción.
- Pantalla de cliente y llamado se comunican por BroadcastChannel; llamado además usa localStorage y eventos del navegador. Requieren contextos compatibles del mismo origen/perfil; no son un canal de red entre terminales.

Consistencia y datos históricos

Las líneas guardan precio base y los adicionales guardan nombre y precio de la venta. El nombre del producto se obtiene por relación, no por copia histórica. Venta, decremento de stock, ingreso, canje y acumulación comparten una transacción Prisma. Aceptar PedidoWeb bloquea la solicitud y registra la venta en esa misma transacción; notificaciones de venta POS se procesan después del commit. Consumo interno y lotes conservan sus brechas propias de atomicidad.

Despliegue previsto, todavía no acreditado



Este diagrama es una propuesta, no evidencia de un despliegue existente. Antes de aplicarlo hay que sustituir URLs localhost del frontend, configurar CORS, secretos y HTTPS, preservar archivos y revisar tareas cron. Docker Compose actual no contiene servicios para API/frontend ni proxy.

Decisiones y consecuencias

Decisión observada	Consecuencia
PostgreSQL y Prisma	Modelo y migraciones explícitos; desplegar migraciones y regenerar cliente

Decisión observada	Consecuencia
Módulos de negocio NestJS	Permite localizar controladores y servicios por función
Estado de sesión en navegador	Facilita navegación, exige proteger ejecución de scripts y manejar expiración
Eventos en memoria	Simple con un proceso; varias réplicas requieren coordinación
Logos en disco local	Requieren volumen persistente y respaldo aparte de PostgreSQL
Tareas dentro del proceso API	Ejecutan mientras API está activa; réplicas pueden duplicarlas
Stock único por ingrediente	No expresa stock por sucursal ni aislamiento por empresa

Fuentes

Backend, arranque, eventos, manifiesto API, manifiesto web y Compose.

Tiendas y domicilios

La ruta pública Next.js /tienda/[slug] consulta TiendaPublicaController. TiendaAdminController aplica JWT y roles a configuración y bandeja. Empresa guarda tiendaSlug único, tiendaConfig y fidelizacionConfig JSON validados por servicio. PedidoWeb guarda contacto de entrega, snapshot del carrito, clave de reintento y estado. La FK única pedidoId establece relación opcional uno a uno con Pedido; sucursalId y repartidorId referencian sucursal y usuario.

La página pública no usa la sesión del personal. El servicio determina empresa desde el slug; usa precios del catálogo, limita solicitudes y expone seguimiento sin datos de dirección o teléfono. El cajero recibe conteo y bandeja por sondeo cada cinco segundos; comprador consulta estado cada diez. La clave compuesta empresald/clave evita solicitudes repetidas y un bloqueo de fila evita aceptaciones duplicadas. El canje descuenta saldo mediante actualización condicional atómica.

La landing comercial es un sitio estático independiente incluido como archivos normales en el repositorio principal. No contiene el backend ni sustituye el despliegue de las tiendas.

05. Modelo entidad-relación

Fuente: esquema Prisma. Modelo físico del código actual, no un diseño idealizado.

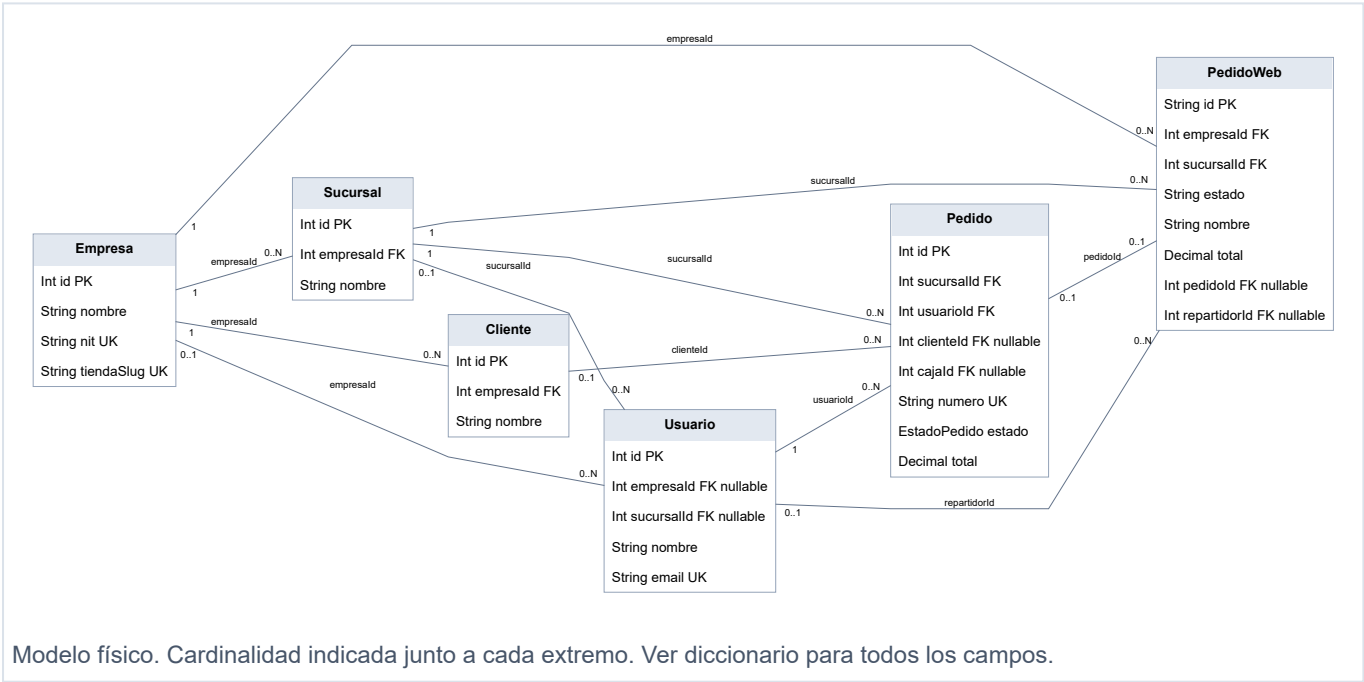
Hay **26 entidades** y **53 relaciones con FK**. Diagrama completo editable. Se muestran PK, FK, unicidad individual y algunos campos descriptivos; el diccionario incluye todos los campos y las restricciones compuestas.

Lectura de cardinalidades

`||` significa exactamente uno; `o|` o `|o`, cero o uno; `o{`, cero o muchos. La FK obligatoria exige un padre por registro hijo, pero no obliga al padre a tener hijos. Las líneas discontinuas señalan relaciones no identificadoras: las tablas tienen PK propia. Los diagramas por dominio omiten relaciones externas al grupo; el completo y la tabla final incluyen todas.

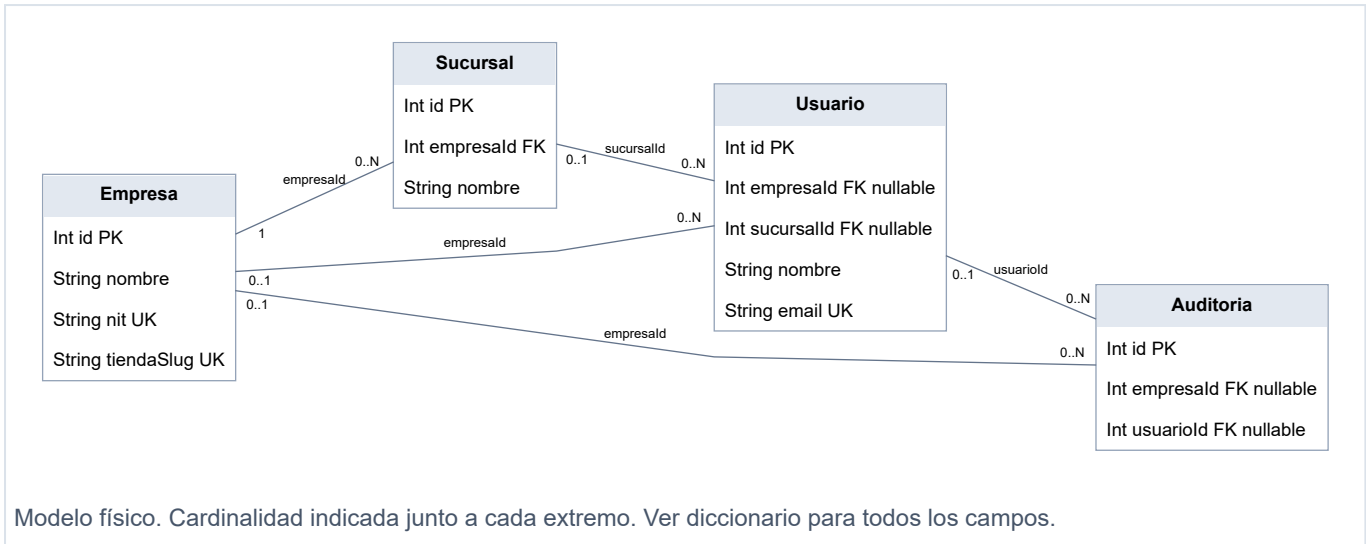
Tienda, domicilios y fidelización

Archivo Mermaid



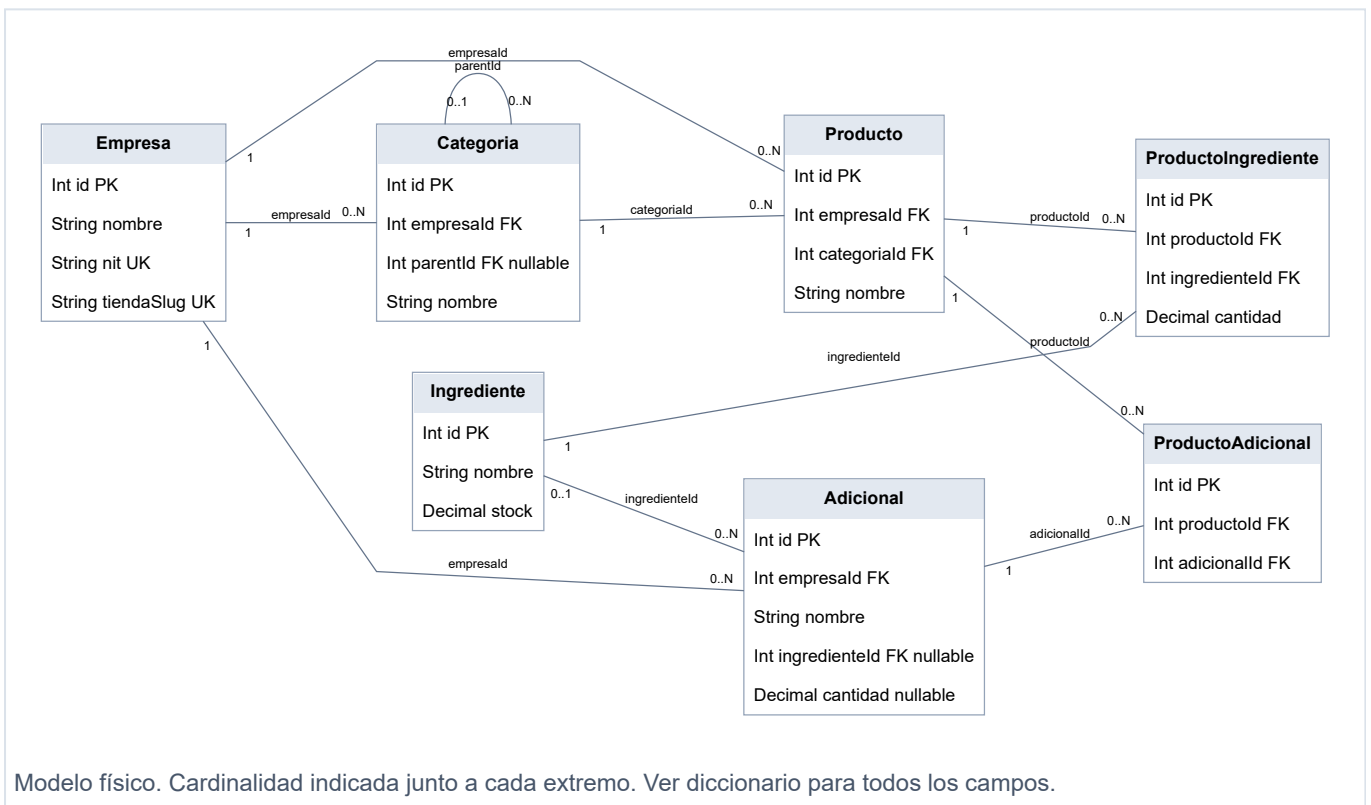
Organización y auditoría

Archivo Mermaid



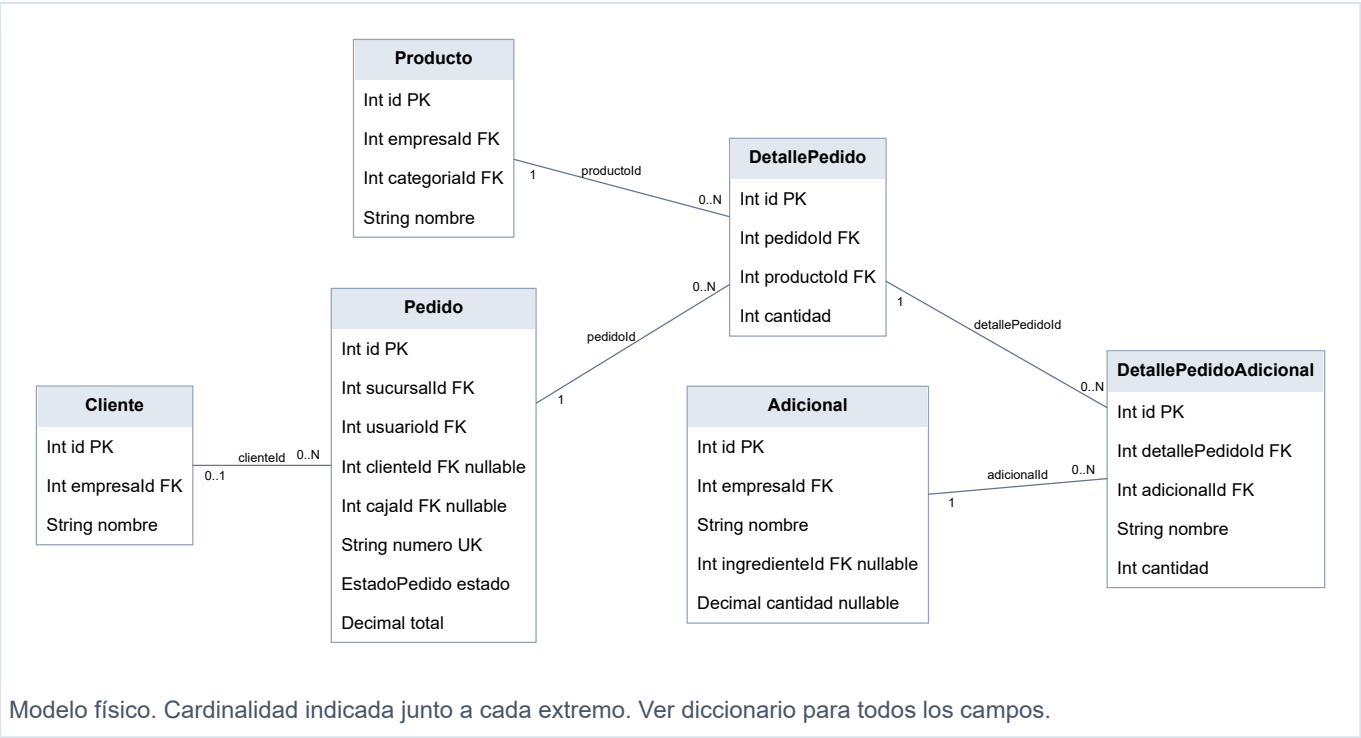
Catálogo, recetas y adicionales

Archivo Mermaid



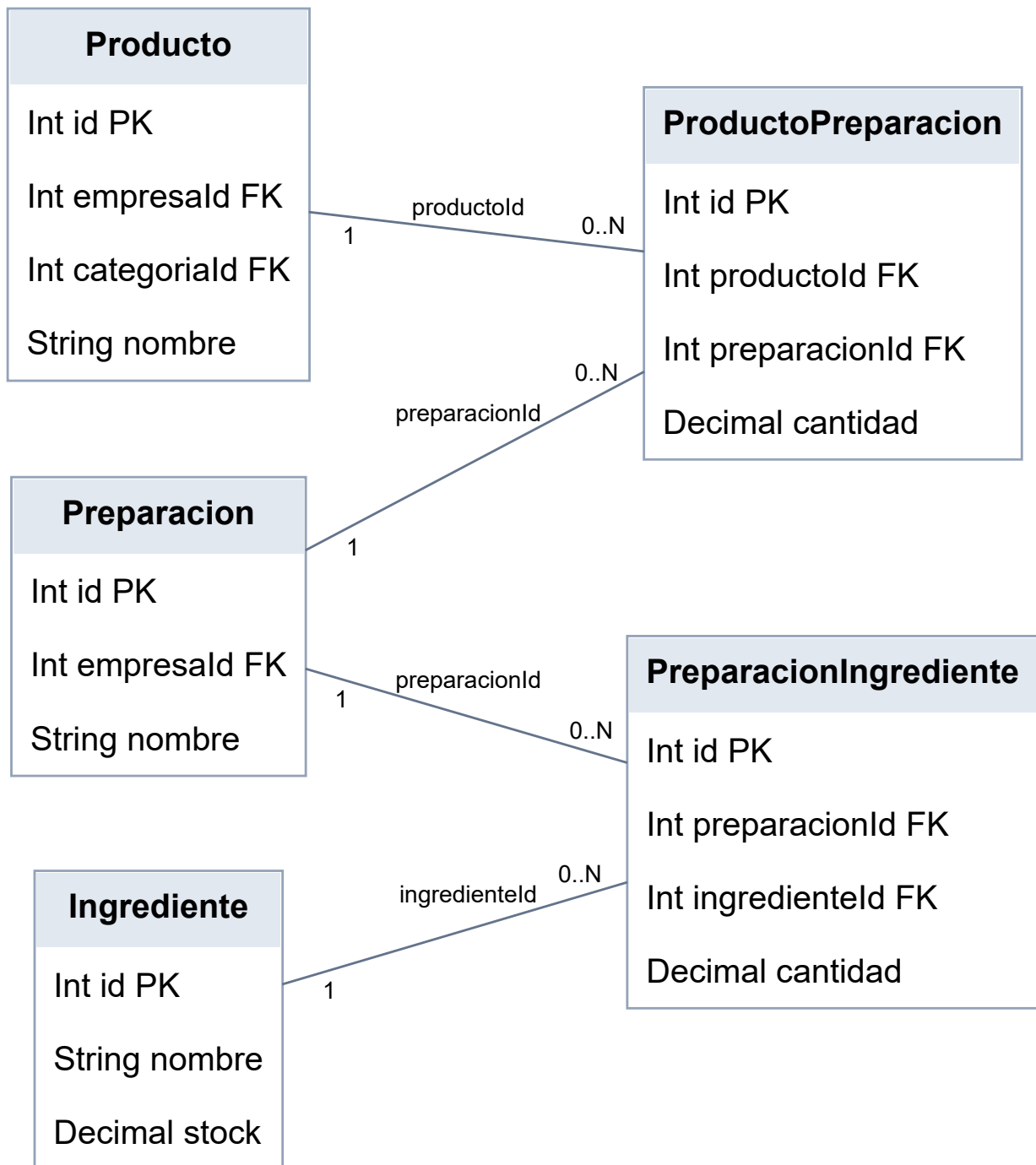
Ventas, clientes y detalles

Archivo Mermaid



Recetas de preparaciones

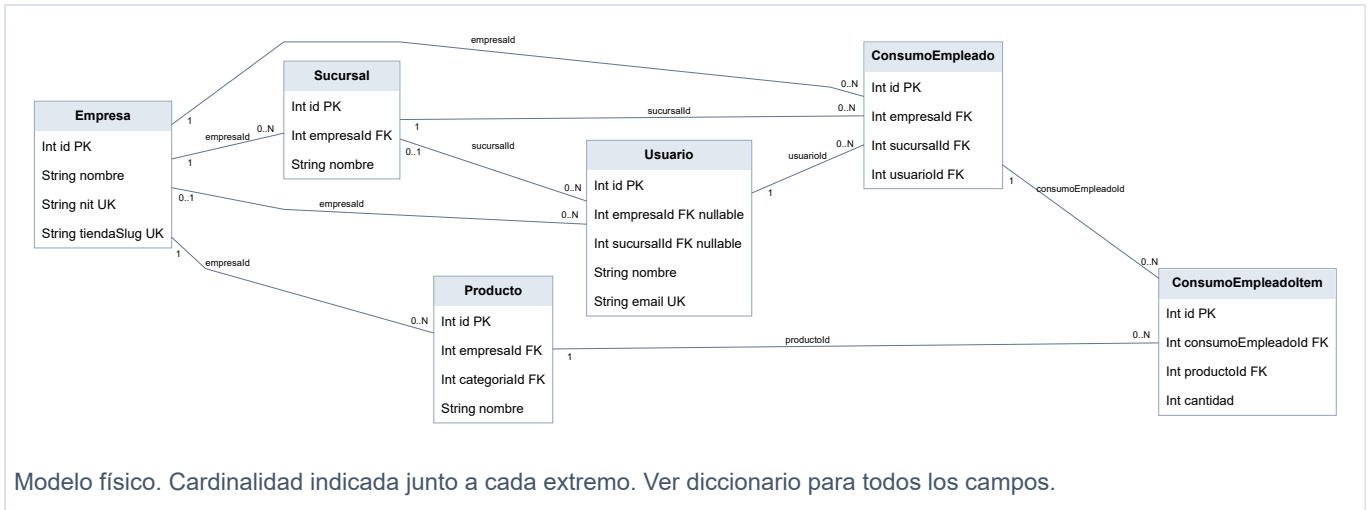
Archivo Mermaid



Modelo físico. Cardinalidad indicada junto a cada extremo. Ver diccionario para todos los campos.

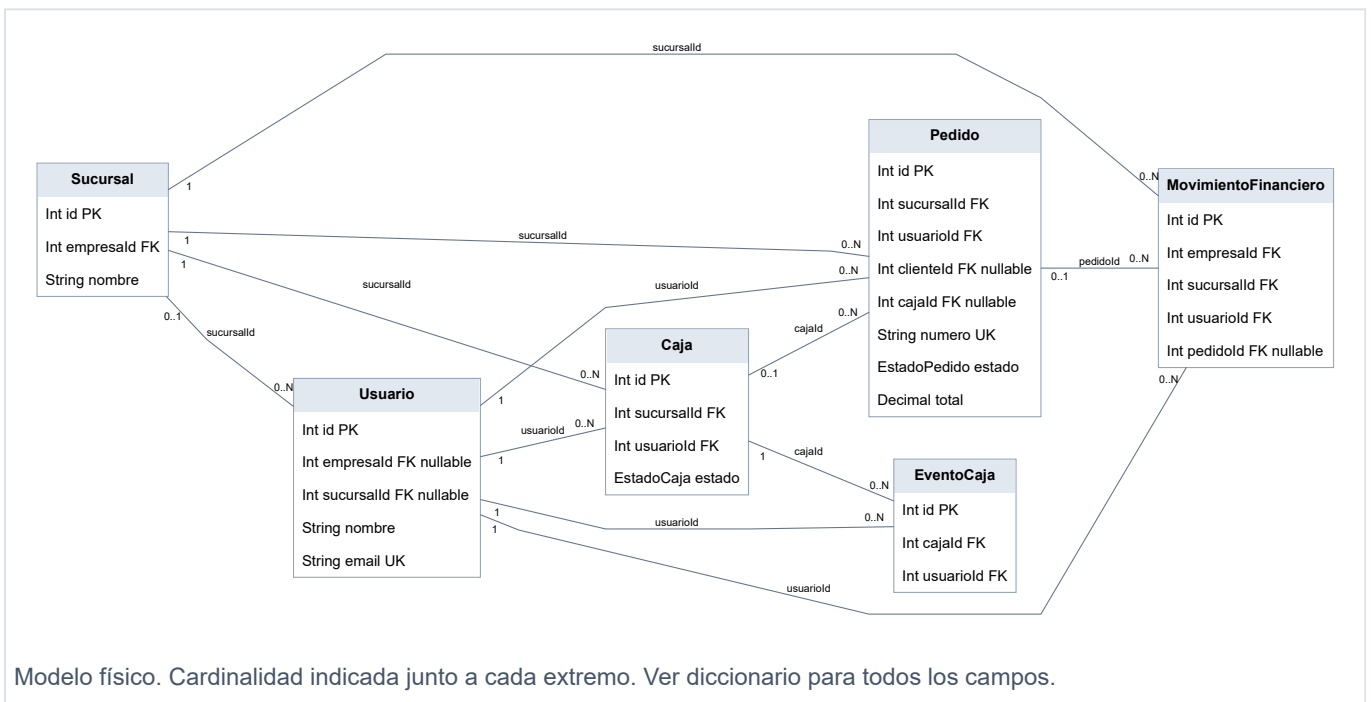
Consumo de empleados

Archivo Mermaid



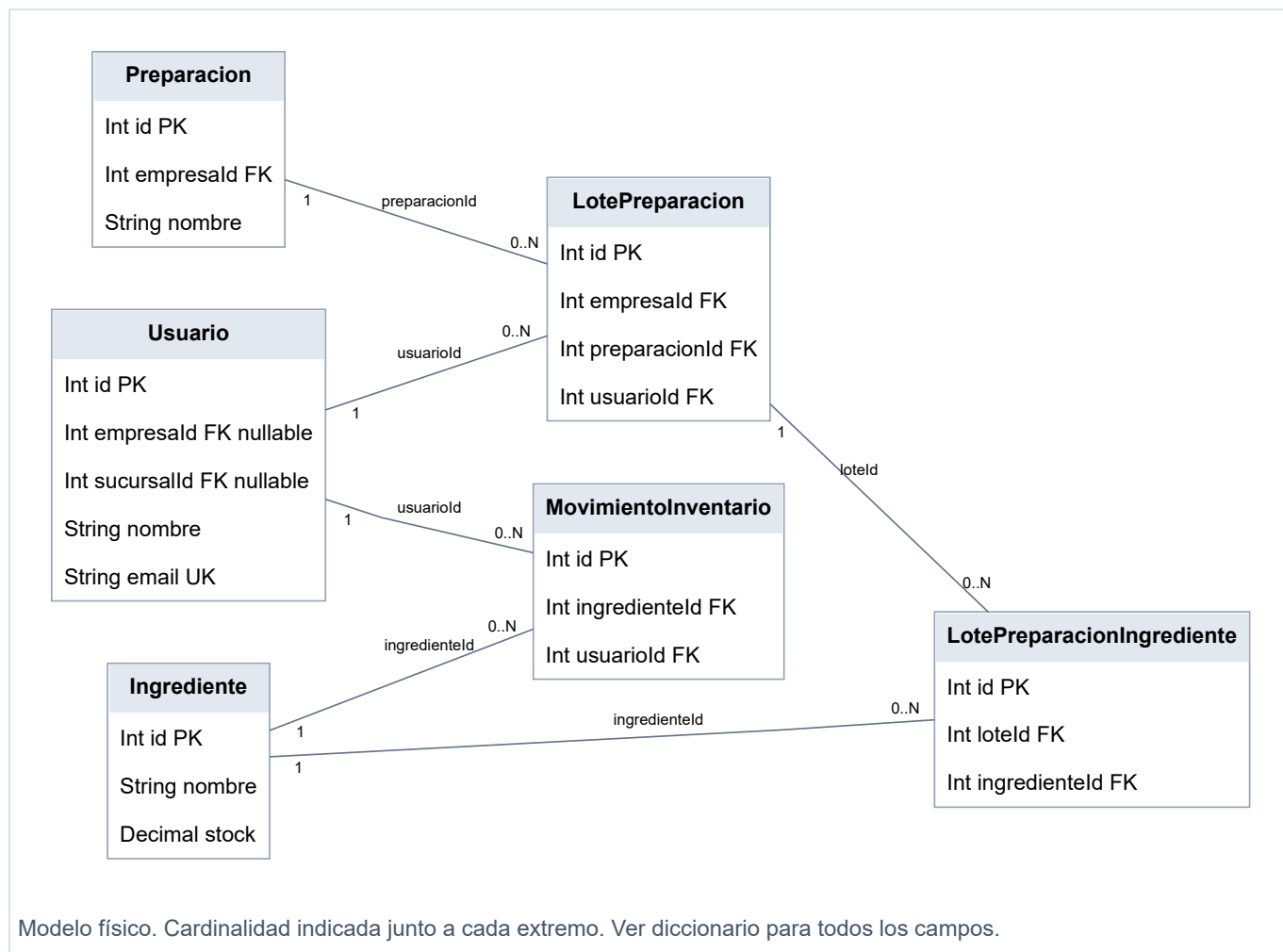
Caja y finanzas

Archivo Mermaid



Lotes y movimientos de inventario

Archivo Mermaid



Relaciones físicas completas

Tabla hija / modelo	FK	Modelo padre / clave	Padre por hijo	Hijos por padre
Sucursal	empresald	Empresa.id	1	0..N
Usuario	empresald	Empresa.id	0..1	0..N
Usuario	sucursald	Sucursal.id	0..1	0..N
Categoria	empresald	Empresa.id	1	0..N
Categoria	parentId	Categoria.id	0..1	0..N
Producto	empresald	Empresa.id	1	0..N
Producto	categoriald	Categoria.id	1	0..N
MovimientoInventario	ingredienteId	Ingrediente.id	1	0..N
MovimientoInventario	usuarioid	Usuario.id	1	0..N
ProductoIngrediente	productoid	Producto.id	1	0..N
ProductoIngrediente	ingredienteId	Ingrediente.id	1	0..N
Preparacion	empresald	Empresa.id	1	0..N

Tabla hija / modelo	FK	Modelo padre / clave	Padre por hijo	Hijos por padre
PreparacionIngrediente	preparacionId	Preparacion.id	1	0..N
PreparacionIngrediente	ingredienteId	Ingrediente.id	1	0..N
LotePreparacion	empresald	Empresa.id	1	0..N
LotePreparacion	preparacionId	Preparacion.id	1	0..N
LotePreparacion	usuarioid	Usuario.id	1	0..N
LotePreparacionIngrediente	loteld	LotePreparacion.id	1	0..N
LotePreparacionIngrediente	ingredienteId	Ingrediente.id	1	0..N
ProductoPreparacion	productoid	Producto.id	1	0..N
ProductoPreparacion	preparacionId	Preparacion.id	1	0..N
Adicional	empresald	Empresa.id	1	0..N
Adicional	ingredienteId	Ingrediente.id	0..1	0..N
ProductoAdicional	productoid	Producto.id	1	0..N
ProductoAdicional	adicionallId	Adicional.id	1	0..N
Cliente	empresald	Empresa.id	1	0..N
Pedido	sucursald	Sucursal.id	1	0..N
Pedido	usuarioid	Usuario.id	1	0..N
Pedido	clienteld	Cliente.id	0..1	0..N
Pedido	cajald	Caja.id	0..1	0..N
DetallePedido	pedidoid	Pedido.id	1	0..N
DetallePedido	productoid	Producto.id	1	0..N
DetallePedidoAdicional	detallePedidoid	DetallePedido.id	1	0..N
DetallePedidoAdicional	adicionallId	Adicional.id	1	0..N
Caja	sucursald	Sucursal.id	1	0..N
Caja	usuarioid	Usuario.id	1	0..N
EventoCaja	cajald	Caja.id	1	0..N
EventoCaja	usuarioid	Usuario.id	1	0..N
MovimientoFinanciero	empresald	Empresa.id	1	0..N
MovimientoFinanciero	sucursald	Sucursal.id	1	0..N
MovimientoFinanciero	usuarioid	Usuario.id	1	0..N
MovimientoFinanciero	pedidoid	Pedido.id	0..1	0..N
ConsumoEmpleado	empresald	Empresa.id	1	0..N
ConsumoEmpleado	sucursald	Sucursal.id	1	0..N
ConsumoEmpleado	usuarioid	Usuario.id	1	0..N

Tabla hija / modelo	FK	Modelo padre / clave	Padre por hijo	Hijos por padre
ConsumoEmpleadoItem	consumoEmpleadoId	ConsumoEmpleado.id	1	0..N
ConsumoEmpleadoItem	productoid	Producto.id	1	0..N
Auditoria	empresald	Empresa.id	0..1	0..N
Auditoria	usuarioid	Usuario.id	0..1	0..N
PedidoWeb	empresald	Empresa.id	1	0..N
PedidoWeb	pedidoid	Pedido.id	0..1	0..1
PedidoWeb	sucursald	Sucursal.id	1	0..N
PedidoWeb	repartidorId	Usuario.id	0..1	0..N

Decisiones y limitaciones del modelo

- Las tablas puente resuelven relaciones muchos a muchos; ProductoIngrediente, ProductoAdicional y ProductoPreparacion impiden pares duplicados mediante claves únicas compuestas.
- Categoria contiene una autorrelación opcional parentId. La FK no evita ciclos jerárquicos por sí misma.
- Usuario puede no tener empresa ni sucursal; esto permite representar SUPERADMIN, aunque el esquema no limita esa excepción a ese rol.
- Ingrediente no tiene empresald ni sucursald: su stock es global. MovimientoInventario tampoco tiene FK directa a empresa o sucursal.
- Pedido pertenece a empresa a través de Sucursal; ConsumoEmpleado y MovimientoFinanciero guardan empresa y sucursal, pero sus FKs independientes no aseguran coincidencia empresarial.
- Auditoria.entidadId es una referencia lógica polimórfica, no una FK a cada entidad auditada.
- ConsumoEmpleado.empleadoNombre es texto; usuarioid identifica quién registra. No existe entidad Empleado.
- DetallePedido.exclusiones es un arreglo de nombres, no una tabla de relación con Ingrediente.
- LotePreparacion no tiene saldo consumido, caducidad ni vínculo de asignación a ventas.
- No hay entidades de factura electrónica, impuestos de línea, pagos múltiples por pedido ni stock por sucursal.
- Activo y disponible expresan estados diferentes; desactivar no equivale a borrar físicamente.
- No se declararon onDelete explícitos en estas relaciones. Este documento no atribuye borrado en cascada; revisar las migraciones antes de cualquier eliminación.

06. Diccionario de datos

Generado desde schema.prisma. Describe el esquema del código, no confirma que una base desplegada haya aplicado sus migraciones.

Incluye **26 modelos**, **53 relaciones físicas** y **9 enumeraciones**.

Convenciones: `?` admite nulo; `[]` es lista; `@id` identifica PK, `@unique` unicidad y `@relation` FK. Campos de relación Prisma no son columnas adicionales. `@updatedAt` lo administra Prisma; no implica un trigger de PostgreSQL. `Decimal(p,s)` conserva precisión y escala declaradas; los servicios pueden convertir valores a Number.

Enumeraciones

Nombre	Valores
PlanEmpresa	BASICO, MEDIUM, PREMIUM
ModoPreparacion	KDS, COMANDAS
RolUsuario	SUPERADMIN, ADMIN_EMPRESA, GERENTE, CAJERO, COCINERO, DOMICILIARIO
EstadoPedido	PENDIENTE, EN_COCINA, LISTO, ENTREGADO, ANULADO
MetodoPago	EFFECTIVO, TARJETA, TRANSFERENCIA, NEQUI, DAVIPLATA
EstadoCaja	ABIERTA, CERRADA
TipoEventoCaja	APERTURA, CIERRE, APERTURA_IRREGULAR, DIFERENCIA, MOVIMIENTO
TipoMovimiento	INGRESO, EGRESO
CategoriaMovimiento	VENTA, COMPRA_INSUMOS, NOMINA, SERVICIOS, ARRIENDO, MANTENIMIENTO, IMPUESTOS, OTROS

Empresa

Tabla física: `empresas`.

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	<code>@id @default(autoincrement())</code>	—
nombre	String	No	—	—
nit	String	No	<code>@unique</code>	—
email	String	No	—	—
telefono	String?	Sí	—	—
direccion	String?	Sí	—	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
logo	String?	Sí	—	—
activo	Boolean	No	@default(true)	—
plan	PlanEmpresa	No	@default(BASICO)	—
permisos	Json	No	@default("{}")	—
modoPreparacion	ModoPreparacion	No	@default(KDS)	—
facturacionElectronicaHabilitada	Boolean	No	@default(false)	—
consumoEmpleadosHabilitado	Boolean	No	@default(false)	—
tiendaSlug	String	No	@unique @default(uuid())	—
tiendaConfig	Json	No	@default("{}")	—
fidelizacionConfig	Json	No	@default("{}")	—
creadoEn	DateTime	No	@default(now())	—
actualizadoEn	DateTime	No	@updatedAt	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
pedidosWeb	PedidoWeb[]	Lado inverso; FK declarada en el otro modelo
usuarios	Usuario[]	Lado inverso; FK declarada en el otro modelo
sucursales	Sucursal[]	Lado inverso; FK declarada en el otro modelo
categorias	Categoria[]	Lado inverso; FK declarada en el otro modelo
productos	Producto[]	Lado inverso; FK declarada en el otro modelo
clientes	Cliente[]	Lado inverso; FK declarada en el otro modelo
movimientos	MovimientoFinanciero[]	Lado inverso; FK declarada en el otro modelo
auditorias	Auditoria[]	Lado inverso; FK declarada en el otro modelo
adicionales	Adicional[]	Lado inverso; FK declarada en el otro modelo
preparaciones	Preparacion[]	Lado inverso; FK declarada en el otro modelo
lotesPreparacion	LotePreparacion[]	Lado inverso; FK declarada en el otro modelo
consumosEmpleados	ConsumoEmpleado[]	Lado inverso; FK declarada en el otro modelo

Sucursal

Tabla física: `sucursales` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresald	Int	No	—; FK a Empresa.id	—
nombre	String	No	—	—
direccion	String?	Sí	—	—
telefono	String?	Sí	—	—
activo	Boolean	No	@default(true)	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
pedidosWeb	PedidoWeb[]	Lado inverso; FK declarada en el otro modelo
empresa	Empresa	@relation(fields: [empresald], references: [id])
usuarios	Usuario[]	Lado inverso; FK declarada en el otro modelo
pedidos	Pedido[]	Lado inverso; FK declarada en el otro modelo
cajas	Caja[]	Lado inverso; FK declarada en el otro modelo
movimientos	MovimientoFinanciero[]	Lado inverso; FK declarada en el otro modelo
consumosEmpleados	ConsumoEmpleado[]	Lado inverso; FK declarada en el otro modelo

Usuario

Tabla física: `usuarios`.

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresald	Int?	Sí	—; FK a Empresa.id	—
sucursald	Int?	Sí	—; FK a Sucursal.id	—
nombre	String	No	—	—
email	String	No	@unique	—
password	String	No	—	—
rol	RolUsuario	No	—	—
permisos	Json?	Sí	—	—
activo	Boolean	No	@default(true)	—
creadoEn	DateTime	No	@default(now())	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
actualizadoEn	DateTime	No	@updatedAt	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
entregasWeb	PedidoWeb[]	Lado inverso; FK declarada en el otro modelo
empresa	Empresa?	@relation(fields: [empresald], references: [id])
sucursal	Sucursal?	@relation(fields: [sucursald], references: [id])
pedidos	Pedido[]	Lado inverso; FK declarada en el otro modelo
cajas	Caja[]	Lado inverso; FK declarada en el otro modelo
eventos	EventoCaja[]	Lado inverso; FK declarada en el otro modelo
movimientos	MovimientoFinanciero[]	Lado inverso; FK declarada en el otro modelo
movimientosInventario	MovimientoInventario[]	Lado inverso; FK declarada en el otro modelo
lotesPreparacion	LotePreparacion[]	Lado inverso; FK declarada en el otro modelo
auditorias	Auditoria[]	Lado inverso; FK declarada en el otro modelo
consumosEmpleados	ConsumoEmpleado[]	Lado inverso; FK declarada en el otro modelo

Categoría

Tabla física: `categorias` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresald	Int	No	—; FK a Empresa.id	—
parentId	Int?	Sí	—; FK a Categoría.id	—
nombre	String	No	—	—
descripcion	String?	Sí	—	—
icono	String?	Sí	—	—
color	String?	Sí	—	—
activo	Boolean	No	@default(true)	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
empresa	Empresa	@relation(fields: [empresald], references: [id])
categoriaPadre	Categoria?	@relation("CategoriaPadre", fields: [parentId], references: [id])
subcategorias	Categoria[]	@relation("CategoriaPadre")
productos	Producto[]	Lado inverso; FK declarada en el otro modelo

Producto

Tabla física: productos .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresald	Int	No	—; FK a Empresa.id	—
categoriald	Int	No	—; FK a Categoria.id	—
nombre	String	No	—	—
descripcion	String?	Sí	—	—
precio	Decimal	No	@db.Decimal(10, 2)	—
imagen	String?	Sí	—	—
disponible	Boolean	No	@default(true)	—
aceptaAdicionales	Boolean	No	@default(true)	—
activo	Boolean	No	@default(true)	—
creadoEn	DateTime	No	@default(now())	—
actualizadoEn	DateTime	No	@updatedAt	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
empresa	Empresa	@relation(fields: [empresald], references: [id])
categoria	Categoria	@relation(fields: [categoriald], references: [id])
ingredientes	ProductoIngrediente[]	Lado inverso; FK declarada en el otro modelo
adicionales	ProductoAdicional[]	Lado inverso; FK declarada en el otro modelo
preparaciones	ProductoPreparacion[]	Lado inverso; FK declarada en el otro modelo
detallesPedido	DetallePedido[]	Lado inverso; FK declarada en el otro modelo
consumoEmpleadoItems	ConsumoEmpleadoItem[]	Lado inverso; FK declarada en el otro modelo

Ingrediente

Tabla física: `ingredientes` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
nombre	String	No	—	—
unidad	String	No	—	—
unidadCompra	String?	Sí	—	—
factorConversion	Decimal?	Sí	@db.Decimal(10, 3)	—
costoUnitario	Decimal?	Sí	@db.Decimal(10, 4)	—
stock	Decimal	No	@db.Decimal(10, 3)	—
stockMinimo	Decimal	No	@db.Decimal(10, 3) @default(0)	—
activo	Boolean	No	@default(true)	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
productos	ProductoIngrediente[]	Lado inverso; FK declarada en el otro modelo
movimientos	MovimientoInventario[]	Lado inverso; FK declarada en el otro modelo
adicionales	Adicional[]	Lado inverso; FK declarada en el otro modelo
preparacionIngredientes	PreparacionIngrediente[]	Lado inverso; FK declarada en el otro modelo
lotePreparacionIngredientes	LotePreparacionIngrediente[]	Lado inverso; FK declarada en el otro modelo

MovimientoInventario

Tabla física: `movimientos_inventario` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
ingredientId	Int	No	—; FK a Ingrediente.id	—
usuarioId	Int	No	—; FK a Usuario.id	—
tipo	String	No	—	—
cantidadMovida	Decimal	No	@db.Decimal(10, 3)	—
unidadCompra	String?	Sí	—	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
factorAplicado	Decimal?	Sí	@db.Decimal(10, 3)	—
stockAnterior	Decimal	No	@db.Decimal(10, 3)	—
stockNuevo	Decimal	No	@db.Decimal(10, 3)	—
descripcion	String?	Sí	—	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
ingrediente	Ingrediente	@relation(fields: [ingredienteId], references: [id])
usuario	Usuario	@relation(fields: [usuarioId], references: [id])

ProductoIngrediente

Tabla física: `producto_ingredientes`.

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
productoid	Int	No	—; FK a Producto.id	—
ingredienteid	Int	No	—; FK a Ingrediente.id	—
cantidad	Decimal	No	@db.Decimal(10, 3)	—

Restricciones de modelo: `@@unique([productoId, ingredienteId])`.

Relaciones de navegación Prisma:

Campo	Destino	Declaración
producto	Producto	@relation(fields: [productoid], references: [id])
ingrediente	Ingrediente	@relation(fields: [ingredienteid], references: [id])

Preparacion

Tabla física: `preparaciones`.

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresaid	Int	No	—; FK a Empresa.id	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
nombre	String	No	—	—
descripcion	String?	Sí	—	—
unidad	String	No	@default("porciones")	—
activo	Boolean	No	@default(true)	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
empresa	Empresa	@relation(fields: [empresald], references: [id])
ingredientes	PreparacionIngrediente[]	Lado inverso; FK declarada en el otro modelo
lotes	LotePreparacion[]	Lado inverso; FK declarada en el otro modelo
productoPreparaciones	ProductoPreparacion[]	Lado inverso; FK declarada en el otro modelo

PreparacionIngrediente

Tabla física: `preparacion_ingredientes` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
preparacionId	Int	No	—; FK a Preparacion.id	—
ingredientId	Int	No	—; FK a Ingrediente.id	—
cantidad	Decimal	No	@db.Decimal(10, 3)	—
unidad	String?	Sí	—	—
observacion	String?	Sí	—	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
preparacion	Preparacion	@relation(fields: [preparacionId], references: [id])
ingrediente	Ingrediente	@relation(fields: [ingredientId], references: [id])

LotePreparacion

Tabla física: `lotes_preparacion` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresald	Int	No	—; FK a Empresa.id	—
preparacionId	Int	No	—; FK a Preparacion.id	—
usuariold	Int	No	—; FK a Usuario.id	—
fecha	DateTime	No	@default(now())	—
loteNumero	String?	Sí	—	—
cantidadProducida	Decimal	No	@db.Decimal(10, 3)	—
pesoTotal	Decimal?	Sí	@db.Decimal(10, 3)	—
pesoPorBolsa	Decimal?	Sí	@db.Decimal(10, 3)	—
porcionesPorBolsa	Decimal?	Sí	@db.Decimal(10, 3)	—
porcionesTotales	Decimal?	Sí	@db.Decimal(10, 3)	—
observacion	String?	Sí	—	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
empresa	Empresa	@relation(fields: [empresald], references: [id])
preparacion	Preparacion	@relation(fields: [preparacionId], references: [id])
usuario	Usuario	@relation(fields: [usuariold], references: [id])
ingredientes	LotePreparacionIngrediente[]	Lado inverso; FK declarada en el otro modelo

LotePreparacionIngrediente

Tabla física: `lote_preparacion_ingredientes` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
loteld	Int	No	—; FK a LotePreparacion.id	—
ingredienteld	Int	No	—; FK a Ingrediente.id	—
cantidadUsada	Decimal	No	@db.Decimal(10, 3)	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
unidad	String?	Sí	—	—
observacion	String?	Sí	—	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
lote	LotePreparacion	@relation(fields: [loteld], references: [id])
ingrediente	Ingrediente	@relation(fields: [ingredienteld], references: [id])

ProductoPreparacion

Tabla física: `producto_preparaciones` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
productold	Int	No	—; FK a Producto.id	—
preparacionId	Int	No	—; FK a Preparacion.id	—
cantidad	Decimal	No	@db.Decimal(10, 3) @default(1)	—
unidad	String	No	@default("porciones")	—
createdAt	DateTime	No	@default(now())	—

Restricciones de modelo: `@@unique([productoId, preparacionId])` .

Relaciones de navegación Prisma:

Campo	Destino	Declaración
producto	Producto	@relation(fields: [productold], references: [id])
preparacion	Preparacion	@relation(fields: [preparacionId], references: [id])

Adicional

Tabla física: `adicionales` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresald	Int	No	—; FK a Empresa.id	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
nombre	String	No	—	—
precio	Decimal	No	@db.Decimal(10, 2)	—
ingredienteId	Int?	Sí	—; FK a Ingrediente.id	—
cantidad	Decimal?	Sí	@db.Decimal(10, 3)	consumo de inventario por 1 unidad de adicional
disponible	Boolean	No	@default(true)	—
activo	Boolean	No	@default(true)	—
creadoEn	DateTime	No	@default(now())	—
actualizadoEn	DateTime	No	@updatedAt	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
empresa	Empresa	@relation(fields: [empresald], references: [id])
ingrediente	Ingrediente?	@relation(fields: [ingredienteId], references: [id])
productos	ProductoAdicional[]	Lado inverso; FK declarada en el otro modelo
detalles	DetallePedidoAdicional[]	Lado inverso; FK declarada en el otro modelo

ProductoAdicional

Tabla física: `producto_adicionales` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
productoid	Int	No	—; FK a Producto.id	—
adicionalId	Int	No	—; FK a Adicional.id	—

Restricciones de modelo: `@@unique([productoId, adicionalId])` .

Relaciones de navegación Prisma:

Campo	Destino	Declaración
producto	Producto	@relation(fields: [productoid], references: [id])
adicional	Adicional	@relation(fields: [adicionalId], references: [id])

Cliente

Tabla física: `clientes` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresald	Int	No	—; FK a Empresa.id	—
nombre	String	No	—	—
documento	String?	Sí	—	—
telefono	String?	Sí	—	—
email	String?	Sí	—	—
direccion	String?	Sí	—	—
fechaNacimiento	DateTime?	Sí	—	—
puntos	Int	No	@default(0)	—
activo	Boolean	No	@default(true)	—
creadoEn	DateTime	No	@default(now())	—

Restricciones de modelo: `@@unique([empresaId, documento])` .

Relaciones de navegación Prisma:

Campo	Destino	Declaración
empresa	Empresa	@relation(fields: [empresald], references: [id])
pedidos	Pedido[]	Lado inverso; FK declarada en el otro modelo

Pedido

Tabla física: `pedidos` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
sucursald	Int	No	—; FK a Sucursal.id	—
usuarioid	Int	No	—; FK a Usuario.id	—
clienteld	Int?	Sí	—; FK a Cliente.id	—
cajald	Int?	Sí	—; FK a Caja.id	—
numero	String	No	@unique	—
estado	EstadoPedido	No	@default(PENDIENTE)	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
metodoPago	MetodoPago	No	@default(EFFECTIVO)	—
subtotal	Decimal	No	@db.Decimal(10, 2)	—
descuento	Decimal	No	@db.Decimal(10, 2) @default(0)	—
total	Decimal	No	@db.Decimal(10, 2)	—
observacion	String?	Sí	—	—
creadoEn	DateTime	No	@default(now())	—
actualizadoEn	DateTime	No	@updatedAt	—
puntosGanados	Int	No	@default(0)	—
puntosCanjeados	Int	No	@default(0)	—
valorPuntoAplicado	Decimal	No	@db.Decimal(10, 2) @default(0)	—
costoDomicilio	Decimal	No	@db.Decimal(10, 2) @default(0)	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
pedidoWeb	PedidoWeb?	Lado inverso; FK declarada en el otro modelo
sucursal	Sucursal	@relation(fields: [sucursalId], references: [id])
usuario	Usuario	@relation(fields: [usuarioId], references: [id])
cliente	Cliente?	@relation(fields: [clienteId], references: [id])
caja	Caja?	@relation(fields: [cajaId], references: [id])
detalles	DetallePedido[]	Lado inverso; FK declarada en el otro modelo
movimientos	MovimientoFinanciero[]	Lado inverso; FK declarada en el otro modelo

DetallePedido

Tabla física: `detalle_pedidos` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
pedidoId	Int	No	—; FK a Pedido.id	—
productId	Int	No	—; FK a Producto.id	—
cantidad	Int	No	—	—
precioUnitario	Decimal	No	@db.Decimal(10, 2)	—
subtotal	Decimal	No	@db.Decimal(10, 2)	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
exclusiones	String[]	No	—	—
observacion	String?	Sí	—	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
pedido	Pedido	@relation(fields: [pedidold], references: [id])
producto	Producto	@relation(fields: [productold], references: [id])
adicionales	DetallePedidoAdicional[]	Lado inverso; FK declarada en el otro modelo

DetallePedidoAdicional

Tabla física: `detalle_pedido_adicionales` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
detallePedidold	Int	No	—; FK a DetallePedido.id	—
adicionalId	Int	No	—; FK a Adicional.id	—
nombre	String	No	—	snapshot del nombre al momento de la venta
precio	Decimal	No	@db.Decimal(10, 2)	snapshot del precio unitario
cantidad	Int	No	@default(1)	unidades del adicional por unidad de producto
subtotal	Decimal	No	@db.Decimal(10, 2)	precio * cantidad * detalle.cantidad

Relaciones de navegación Prisma:

Campo	Destino	Declaración
detalle	DetallePedido	@relation(fields: [detallePedidold], references: [id])
adicional	Adicional	@relation(fields: [adicionalId], references: [id])

Caja

Tabla física: `cajas` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
sucursalId	Int	No	—; FK a Sucursal.id	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
usuarioid	Int	No	—; FK a Usuario.id	—
estado	EstadoCaja	No	@default(ABIERTA)	—
montoInicial	Decimal	No	@db.Decimal(10, 2)	—
montoFinal	Decimal?	Sí	@db.Decimal(10, 2)	—
diferencia	Decimal?	Sí	@db.Decimal(10, 2)	—
abiertaEn	DateTime	No	@default(now())	—
cerradaEn	DateTime?	Sí	—	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
sucursal	Sucursal	@relation(fields: [sucursald], references: [id])
usuario	Usuario	@relation(fields: [usuarioid], references: [id])
pedidos	Pedido[]	Lado inverso; FK declarada en el otro modelo
eventos	EventoCaja[]	Lado inverso; FK declarada en el otro modelo

EventoCaja

Tabla física: `eventos_caja`.

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
cajald	Int	No	—; FK a Caja.id	—
usuarioid	Int	No	—; FK a Usuario.id	—
tipo	TipoEventoCaja	No	—	—
descripcion	String	No	—	—
esAlerta	Boolean	No	@default(false)	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
caja	Caja	@relation(fields: [cajald], references: [id])
usuario	Usuario	@relation(fields: [usuarioid], references: [id])

MovimientoFinanciero

Tabla física: `movimientos_financieros` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresald	Int	No	—; FK a Empresa.id	—
sucursald	Int	No	—; FK a Sucursal.id	—
usuariold	Int	No	—; FK a Usuario.id	—
tipo	TipoMovimiento	No	—	—
categoria	CategoriaMovimiento	No	—	—
descripcion	String	No	—	—
monto	Decimal	No	@db.Decimal(10, 2)	—
fecha	DateTime	No	@default(now())	—
pedidold	Int?	Sí	—; FK a Pedido.id	—
comprobante	String?	Sí	—	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
empresa	Empresa	@relation(fields: [empresald], references: [id])
sucursal	Sucursal	@relation(fields: [sucursald], references: [id])
usuario	Usuario	@relation(fields: [usuariold], references: [id])
pedido	Pedido?	@relation(fields: [pedidold], references: [id])

ConsumoEmpleado

Tabla física: `consumos_empleados` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresald	Int	No	—; FK a Empresa.id	—
sucursald	Int	No	—; FK a Sucursal.id	—
usuariold	Int	No	—; FK a Usuario.id	—
empleadoNombre	String	No	—	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
observacion	String?	Sí	—	—
creadoEn	DateTime	No	@default(now())	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
empresa	Empresa	@relation(fields: [empresald], references: [id])
sucursal	Sucursal	@relation(fields: [sucursald], references: [id])
usuario	Usuario	@relation(fields: [usuariold], references: [id])
items	ConsumoEmpleadoItem[]	Lado inverso; FK declarada en el otro modelo

ConsumoEmpleadoItem

Tabla física: `consumo_empleado_items` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
consumoEmpleadold	Int	No	—; FK a ConsumoEmpleado.id	—
productold	Int	No	—; FK a Producto.id	—
cantidad	Int	No	—	—
precioReferencia	Decimal	No	@db.Decimal(10, 2)	—

Relaciones de navegación Prisma:

Campo	Destino	Declaración
consumo	ConsumoEmpleado	@relation(fields: [consumoEmpleadold], references: [id])
producto	Producto	@relation(fields: [productold], references: [id])

Auditoria

Tabla física: `auditorias` .

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	Int	No	@id @default(autoincrement())	—
empresald	Int?	Sí	—; FK a Empresa.id	—
usuariold	Int?	Sí	—; FK a Usuario.id	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
accion	String	No	—	—
entidad	String	No	—	—
entidadId	Int?	Sí	—	—
detalle	Json?	Sí	—	—
creadoEn	DateTime	No	@default(now())	—

Restricciones de modelo: `@@index([empresaId, creadoEn])`.

Relaciones de navegación Prisma:

Campo	Destino	Declaración
empresa	Empresa?	@relation(fields: [empresald], references: [id])
usuario	Usuario?	@relation(fields: [usuarioid], references: [id])

PedidoWeb

Tabla física: `pedidos_web`.

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
id	String	No	@id @default(uuid())	—
empresald	Int	No	—; FK a Empresa.id	—
sucursald	Int	No	—; FK a Sucursal.id	—
clave	String	No	—	—
estado	String	No	@default("RECIBIDO")	—
nombre	String	No	—	—
telefono	String	No	—	—
direccion	String	No	—	—
zona	String	No	—	—
observacion	String?	Sí	—	—
items	Json	No	—	—
subtotal	Decimal	No	@db.Decimal(10, 2)	—
costoDomicilio	Decimal	No	@db.Decimal(10, 2)	—
total	Decimal	No	@db.Decimal(10, 2)	—
metodoPago	MetodoPago	No	@default(EFECTIVO)	—
pedidoId	Int?	Sí	@unique; FK a Pedido.id	—

Campo	Tipo Prisma	Admite nulo	Restricciones / valor inicial	Nota del esquema
repartidorId	Int?	Sí	—; FK a Usuario.id	—
motivo	String?	Sí	—	—
creadoEn	DateTime	No	@default(now())	—
actualizadoEn	DateTime	No	@updatedAt	—

Restricciones de modelo: `@@unique([empresaId, clave]); @@index([empresaId, estado, creadoEn])`.

Relaciones de navegación Prisma:

Campo	Destino	Declaración
empresa	Empresa	@relation(fields: [empresald], references: [id])
pedido	Pedido?	@relation(fields: [pedidold], references: [id])
sucursal	Sucursal	@relation(fields: [sucursald], references: [id])
repartidor	Usuario?	@relation(fields: [repartidorId], references: [id])

07. Referencia de API

Convenciones

Base local: `http://localhost:3000`. No hay prefijo global `/api` en `main.ts`. Cuerpos normalmente JSON con `Content-Type: application/json`; logo usa multipart. Los ejemplos usan IDs ilustrativos que deben sustituirse por registros del ambiente de prueba.

Enviar `Authorization: Bearer <TOKEN>` a rutas protegidas. Login y registro público de empresa no declaran guard. `/` es una ruta de ejemplo. Las respuestas de error de NestJS suelen incluir `statusCode`, `message` y `error`, pero no existe un contrato uniforme documentado para errores de Prisma no interceptados.

Catálogo completo de rutas: generado desde todas las clases de controlador. Incluye guards de clase, roles y parámetros anotados.

Autenticación

POST `/auth/login`

```
{
  "email": "operador@example.invalid",
  "password": "CONTRASENA_DEL_AMBIENTE_DE_PRUEBA"
}
```

Respuesta normal: `access_token` y `usuario` con identidad, rol, empresa/sucursal y configuración empresarial. Contraseña incorrecta, usuario inactivo o empresa inactiva se rechazan con 401. Vigencia declarada del JWT: ocho horas.

POST `/auth/registrar-empresa`

```
{
  "empresa": { "nombre": "Negocio de prueba", "nit": "NIT-PRUEBA-001", "email": "negocio@example.invalid" },
  "admin": { "nombre": "Administrador de prueba", "email": "admin@example.invalid", "password":
    "CONTRASENA_TEMPORAL_UNICA" }
}
```

Devuelve mensaje, datos de empresa y sucursal. Los duplicados identificados se rechazan con 409. La creación de administrador es posterior a la de empresa; no se garantiza reversión integral en caso de error.

Caja y venta

POST `/caja/abrir`

```
{ "montoInicial": 50000, "cajeroId": 7 }
```

La sucursal proviene del token, con respaldo actual a ID 1 cuando falta. Monto no negativo y cajero activo son condiciones comprobadas; el ID 7 es ficticio. Rechaza caja ya abierta.

POST /pedidos

```
{
  "sucursalId": 2,
  "clienteId": 12,
  "metodoPago": "EFECTIVO",
  "descuento": 0,
  "observacion": "Para llevar",
  "items": [
    {
      "productoId": 31,
      "cantidad": 2,
      "exclusiones": ["Cebolla"],
      "observacion": "Sin salsa",
      "adicionales": [{ "adicionalId": 4, "cantidad": 1 }]
    }
  ]
}
```

clienteId, observacion, descuento, adicionales y exclusiones pueden omitirse según la lógica de servicio. cajaId es opcional y se resuelve desde la caja abierta. Si se envía explícitamente, hay una comprobación adicional de pertenencia al usuario que puede afectar a supervisores.

El servidor toma precios del catálogo y crea estado PENDIENTE. Devuelve pedido con detalles, adicionales y datos parciales de usuario/cliente. La cantidad y el rango del descuento no tienen validación robusta centralizada. No enviar importes calculados por el cliente como fuente autoritativa.

GET /pedidos?sucursalId=2 devuelve como máximo 50 pedidos recientes, filtrados por empresa y opcionalmente sucursal. No hay paginación general en esta ruta.

PATCH /pedidos/:id/estado

```
{ "estado": "LISTO" }
```

Las solicitudes web RECIBIDO se pueden rechazar sin venta. Una venta web aceptada o con movimientos de puntos bloquea la anulación directa y requiere conciliación. No existe todavía un flujo automático de devolución que revierta ingreso e inventario; no presentar ANULADO como reembolso. Los estados de preparación siguen disponibles; un pedido anulado no puede reabrirse.

POST /caja/:id/cerrar

```
{ "montoFinal": 92000 }
```

Devuelve caja actualizada, ventas, esperado, diferencia y marca de automático. El monto esperado incluye ventas de todos los métodos. No usarlo como saldo exclusivo de billetes y monedas.

Inventario

POST /inventario

```
{ "nombre": "Insumo de prueba", "unidad": "gramos", "stock": 1000, "stockMinimo": 200 }
```

POST /inventario/:id/ajuste

```
{ "tipo": "ENTRADA", "cantidad": 500, "enUnidadCompra": false, "descripcion": "Recepción de prueba" }
```

ENTRADA suma, SALIDA resta; la rama alternativa del servicio sustituye el saldo. Usar AJUSTE para el caso previsto, sin asumir que otros valores serán rechazados. Historial en `GET /inventario/:id/historial`, máximo 50 registros. El inventario no se filtra por empresa.

Preparaciones y consumo interno

POST /preparaciones/:id/lotos

```
{
  "loteNumero": "LOTE-PRUEBA-001",
  "cantidadProducida": 10,
  "porcionesTotales": 10,
  "observacion": "Producción de prueba",
  "ingredientes": [{ "ingredienteId": 5, "cantidadUsada": 500, "unidad": "gramos" }]
}
```

Se requieren producción y porciones positivas. Los insumos usados se toman del cuerpo. El saldo de porciones mostrado no considera consumos por venta.

POST /consumo-empleados

```
{
  "sucursalId": 2,
  "empleadoNombre": "Empleado de prueba",
  "observacion": "Alimentación del turno",
  "items": [{ "productoId": 31, "cantidad": 1 }]
}
```

Solo ADMIN_EMPRESA/GERENTE; función habilitada en empresa. Devuelve consumo con líneas y referencias. No crea ingreso ni pedido de venta. `GET /consumo-empleados/resumen` ofrece totales mensuales según controlador.

SSE

`GET /pedidos/stream?token=<TOKEN>` abre `text/event-stream`. La aplicación utiliza token en query para EventSource. El dato de evento tiene forma:

```
{ "tipo": "CREADO", "pedidoId": 123, "estado": "PENDIENTE" }
```

También se emite `ACTUALIZADO`. El canal filtra por empresa, vive en memoria y no ofrece historial persistente. Evitar registrar URLs con tokens en logs de despliegue. No se observó un guard de rol específico para stream.

Archivos e impresión

- `POST /empresa/logo` : multipart, campo `logo`, límite 2 MiB, MIME `jpg/jpeg/png/gif/webp`. Respuesta `{ "logoUrl": "..."` }. El archivo se publica bajo `/uploads/logos/`.
- `POST /impresion/comanda` y `/impresion/ticket` : reciben una representación del pedido para imprimir; no basta con asumir que el servidor recibe únicamente un ID.
- `POST /impresion/abrir-cajon` : solicita pulso al hardware configurado.

Errores que debe manejar un cliente

Código	Casos observados o esperados por excepción
400	Caja cerrada/duplicada, producto indisponible, adicional inválido, lote sin cantidad válida
401	Credenciales inválidas o token no válido
403	Rol restringido o consumo de empleados deshabilitado
404	Entidad no encontrada en servicios que lanzan <code>NotFoundException</code>
409	Duplicado de NIT o correo en registro de empresa
500	Fallos no controlados; no interpretar como garantía de que nada se guardó

No se identificó idempotencia para ventas, versionado de API ni documentación OpenAPI. Un error de red al vender requiere consultar el último pedido antes de reintentar. Los cuerpos con `any` no constituyen contratos validados en tiempo de ejecución.

API de tiendas

Método y ruta	Acceso	Contrato principal
GET <code>/tiendas/:slug</code>	Público	Marca, configuración pública y catálogo de la empresa activa.
POST <code>/tiendas/:slug/pedidos</code>	Público limitado	clave, nombre, telefono, direccion, zona, metodoPago, observacion, items[{productold,cantidad}], totalEsperado. Devuelve id, estado, total.
GET <code>/tiendas/:slug/pedidos/:id</code>	Público con identificador de seguimiento	Estado, total, motivo y número/estado de venta; no dirección, teléfono o saldo.
GET <code>/tienda-admin/configuracion</code>	Administrador, gerente, cajero	slug, tienda y fidelizacion de empresa autenticada.

Método y ruta	Acceso	Contrato principal
PATCH /tienda-admin/configuracion	ADMIN_EMPRESA	Actualización parcial de slug, tienda o fidelizacion; valida propiedad, tipos y límites.
GET /tienda-admin/resumen	Administrador, gerente, cajero	Conteo pendientes; cajero limitado a su sucursal.
GET /tienda-admin/pedidos	Administrador, gerente, cajero, domiciliario	Últimos 100 registros del ámbito autorizado; domiciliario solo asignados.
GET /tienda-admin/repartidores	Administrador, gerente, cajero	Usuarios DOMICILIARIO activos de empresa.
PATCH /tienda-admin/pedidos/:id	Roles autorizados según acción	accion: ACEPTAR, RECHAZAR, EN_CAMINO o ENTREGADO; clienteld opcional verificado, motivo o repartidorId según acción.

POST /pedidos admite puntosCanjeados; el servidor calcula descuento y puntosGanados. Pedido conserva puntosGanados, puntosCanjeados, valorPuntoAplicado y costoDomicilio. La ruta anterior POST /clientes/:id/puntos/redimir rechaza con instrucción de canjear en POS; ajustes positivos se reservan a ADMIN_EMPRESA.

Errores principales: 400 por entrada inválida, tienda pausada, caja cerrada o saldo insuficiente; 401 por sesión ausente; 403 por rol/recurso no autorizado; 404 por empresa/solicitud no disponible en el ámbito; 409 por enlace repetido, precio cambiado o revisión repetida; 429 por límite de IP. Sin transacción externa de pago.

08. Instalación, despliegue y operación

Alcance de esta guía

Procedimiento basado en archivos y scripts existentes. En la revisión documental 1.1 se ejecutaron las migraciones y una restauración en bases temporales con datos sintéticos; no se desplegó el producto ni se aplicó seed sobre datos reales. Ver el informe de validación. Los comandos de desarrollo deben ejecutarse en un ambiente separado de datos de cliente.

Requisitos y estructura

Node.js compatible con los manifests (el README indica 20+; la herramienta local inspeccionada reportó 24.19.0), npm, Docker con Compose y navegador. API y frontend tienen sus propios `package.json` y lockfiles; la raíz no tiene script de arranque conjunto y declara pnpm mediante devEngines. Los pasos siguientes usan npm desde cada subproyecto. Acordar un gestor y conservar sus lockfiles en el equipo.

Puertos locales: PostgreSQL 5432, Redis 6379, API 3000 y frontend 3001. Redis está en Compose, pero no se identificó consumo desde los servicios revisados.

1. Servicios locales

Desde la raíz del repositorio:

```
docker compose up -d
docker compose ps
```

Compose crea volúmenes para PostgreSQL y Redis. La configuración incluida contiene credenciales de desarrollo; no reutilizarlas en el ambiente entregado a compradores.

2. Configuración de API

Crear `api/.env` local, sin confirmarlo en Git. Plantilla sin secretos reales:

```
DATABASE_URL="postgresql://USUARIO:CLAVE@localhost:5432/BASE?schema=public"
JWT_SECRET="REEMPLAZAR_POR_SECRETO_ALEATORIO_PROPIO"
APP_URL="http://localhost:3000"
ESC_POS_ENABLED="false"
ESC_POS_BROWSER_FALLBACK="true"
```

Reemplazar usuario, clave y base por los del servicio PostgreSQL utilizado. `APP_URL` construye URL de logos; no configura la URL Axios del frontend. El código tiene un secreto JWT de respaldo: la entrega debe exigir configurar uno propio.

Variable	Finalidad	Necesidad
DATABASE_URL	Conexión PostgreSQL	Obligatoria
JWT_SECRET	Firma y validación JWT	Obligatoria para entrega segura
APP_URL	URL pública de API para logos	Configurar según ambiente
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS	Correo saliente	Opcional; puerto por defecto 587
EMAIL_ADMIN, ADMIN_PHONE	Destinatarios de respaldo	Opcionales, revisar por empresa
TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN	Autenticación de proveedor	Opcionales
TWILIO_WHATSAPP_PHONE, TWILIO_SMS_PHONE, TWILIO_PHONE	Remitentes y respaldo de remitente	Opcionales
ESC_POS_ENABLED	Habilita impresión directa	true/false
ESC_POS_TYPE	Transporte de impresión	tcp por defecto
ESC_POS_HOST, ESC_POS_PORT	Dirección de impresora	Puerto por defecto 9100
ESC_POS_BROWSER_FALLBACK	Alternativa del navegador	true/false
SUPERADMIN_EMAIL, SUPERADMIN_PASSWORD, SUPERADMIN_NAME	Aprovisionamiento por script	Solo cuando se use crear-superadmin

No se observó uso de variable `PORT` : `main.ts` escucha en 3000. Las variables de integración no sustituyen la configuración o habilitación de cuentas externas.

3. Dependencias, esquema y arranque

En una terminal desde la raíz:

```
cd api
npm ci
npx prisma generate
npx prisma migrate deploy
npm run start:dev
```

`npm ci` requiere manifiesto y lockfile coherentes. Si falla, revisar la discrepancia con el equipo; no regenerar el lockfile como parte rutinaria de una entrega. Aplicar migraciones solo después de confirmar base destino y respaldo. Los cambios locales del esquema podrían requerir nuevas migraciones; `migrate deploy` aplica archivos existentes, no genera los faltantes.

Para cargar una demostración aislada, revisar previamente `api/prisma/seed.ts` y ejecutar `npm run seed` desde `api`. No aplicar seed de demostración sobre datos de cliente. Para crear el superadministrador, configurar las variables indicadas y ejecutar `npm run crear-superadmin`; el script puede actualizar una cuenta existente con ese correo, incluido su rol y contraseña.

4. Frontend

En otra terminal desde la raíz:

```
cd web
npm ci
npm run dev -- --port 3001
```

Abrir `http://localhost:3001/login` . Se indica el puerto explícitamente porque `next dev` no fija 3001 en el script; así se evita colisión con la API.

5. Verificación local

Comprobar acceso al login, autenticación con cuenta de prueba, carga de catálogo, caja y una venta de prueba con control de stock y movimiento financiero. Probar cocina en otra ventana y detenerse ante cualquier inconsistencia. La ruta `/` de API devuelve el ejemplo Hello World; no es una comprobación completa de salud de base de datos.

Para revisar migraciones: `npx prisma migrate status` desde api. Para verificar compilación: `npm run build` en api y luego en web. Desde api, `npm test -- --runInBand` ejecuta pruebas unitarias; `npm run test:e2e -- --runInBand` requiere un ambiente aislado porque importa módulos y tareas del sistema. `npm run lint` de API usa `--fix` y puede modificar archivos.

Preparación del despliegue

Antes de publicar una instancia para adquirentes, completar brechas y registrar:

Elemento	Dato o evidencia requerida
Alojamiento	Proveedor, responsable, región y recursos; por definir
URL	Dominio de web/API y HTTPS; por definir
Red del cliente	Acceso a impresora desde el proceso backend; un servidor remoto no alcanza automáticamente una impresora privada
URLs frontend	Sustituir referencias localhost en Axios, SSE y otras rutas; hoy no hay configuración central por ambiente
CORS	Restringir a orígenes autorizados; hoy usa <code>*</code>
Persistencia	Base de datos y uploads conservados entre versiones
Secretos	Credenciales propias, sin defaults ni cuentas demo
Hora	Zona horaria y horarios de cierre acordados y probados
Instancias	Considerar eventos en memoria y cron antes de aumentar réplicas
Aceptación	Pruebas críticas ejecutadas, incidencias documentadas y acta firmada

Compilación prevista: `npm run build` en ambos proyectos; arranque de API mediante `npm run start:prod` y de web mediante `npm run start -- --port 3001` . Confirmar que la salida del build coincide con el script de arranque antes de instalar un servicio. No hay un procedimiento de CI/CD del proyecto acreditado por esta revisión.

Respaldo y recuperación

Respaldo PostgreSQL y `api/uploads` de forma coordinada. El volumen Docker es persistencia, no una copia externa. La presencia de un SQL en `backups` no demuestra periodicidad ni recuperabilidad.

Ejemplo para el Compose local, desde raíz, sin redirigir binarios a través de PowerShell:

```
New-Item -ItemType Directory -Force backups
docker exec powerpos_db pg_dump -U powerpos -d powerpos_dev -Fc -f /tmp/powerpos-respaldo.dump
docker cp powerpos_db:/tmp/powerpos-respaldo.dump backups/powerpos-respaldo.dump
```

Usar nombre único por fecha en operación para no sobrescribir versiones anteriores, proteger acceso al respaldo y copiarlo fuera del host. El dump puede contener datos personales y hashes de usuarios. Las credenciales de la instancia real pueden diferir del ejemplo.

La restauración debe probarse en **una base nueva y aislada**, no sobre la productiva. Ejemplo para un nombre de prueba que todavía no exista:

```
docker cp backups/powerpos-respaldo.dump powerpos_db:/tmp/powerpos-restaurar.dump
docker exec powerpos_db createdb -U powerpos powerpos_restore_test
docker exec powerpos_db pg_restore -U powerpos -d powerpos_restore_test --no-owner /tmp/powerpos-restaurar.dump
```

Si la base de prueba ya existe, detenerse y elegir otra base; no borrarla automáticamente. Validar conteos, relaciones, login de prueba y lectura de pedidos usando una API aislada, con notificaciones y cron controlados. Registrar fecha, duración, responsable y resultado. La meta de respaldo diario y recuperación en cuatro horas es propuesta, no un compromiso contratado.

Actualización y reversión

1. Registrar versión exacta, cambios, respaldo y ventana de mantenimiento.
2. Probar migraciones y compilación en ambiente aislado.
3. Aplicar versión y ejecutar pruebas de humo.
4. Si falla, detener nuevas escrituras y evaluar restauración con responsable. Revertir código no deshace automáticamente migraciones ni datos.
5. Registrar incidente, cambios de versión y comunicación acordada con cliente.

Diagnóstico

Síntoma	Revisar
Login falla	API, conexión DB, usuario/empresa activos, credenciales y JWT_SECRET
API inaccesible desde otro equipo	localhost del navegador apunta a ese equipo; revisar URL y red
POS bloqueado	Caja abierta, sucursal y cajero asignado
Caja se cierra sola	Cron cada cinco minutos, hora local y horario codificado
Cocina tarda	SSE, token, API y consulta de respaldo de cinco segundos

Síntoma	Revisar
Llamado no se ve en otra terminal	BroadcastChannel/storage no sincronizan terminales distintos
No imprime	Habilitación, transporte TCP, host/puerto, conectividad desde API y alternativa del navegador
Logo roto	APP_URL, archivo persistente y ruta <code>/uploads/</code>
Error tras vender	Consultar pedido, stock e ingreso antes de volver a enviar

Responsables por asignar

Proveedor: alojamiento, actualizaciones, soporte y copias según contrato. Cliente: usuarios autorizados, catálogo, equipos y conteo físico. La distribución final, los horarios y los tiempos de respuesta deben quedar en el acta/contrato de entrega; no se asumen aquí.

Activación de tiendas y fidelización (1.2)

Consultar guía de activación. Migración 20260916070000_tienda_domicilios_fidelizacion aplicada y verificada primero en prueba aislada y luego en powerpos_dev local, respaldada previamente en backups/powerpos_pre_tienda_20260917.dump. No hay despliegue público del sistema. En otra instalación: respaldo, prisma migrate deploy, prisma generate, compilar API/web y reiniciar procesos. Configurar NEXT_PUBLIC_API_URL antes de compilar web; debe ser una URL HTTPS accesible desde los dispositivos de compradores. Verificar URLs públicas de logos/portadas y persistencia de uploads. Activar sucursal, zonas y recepción desde Mi tienda; definir fidelización desde Puntos.

API y frontend compilaron en revisión 1.2; los dos errores de tipos documentados en 1.1 fueron corregidos. Una compilación exitosa no sustituye la prueba del despliegue ni la aprobación del piloto.

09. Manual de usuario

Versión documental 1.2 · Corte 2026-09-15. Para la copia destinada a adquirentes, ver manual del cliente. Este manual describe pantallas encontradas en el código; las funciones disponibles dependen de la configuración entregada y de las correcciones pendientes.

Pantallas

Ruta	Uso
/login	Acceso al sistema
/pos	Registro de pedidos
/cocina	Preparación y estado de pedidos
/dashboard	Resumen operativo, seguimiento y caja
/productos	Categorías, productos, recetas y adicionales
/inventario	Ingredientes, stock, historial y preparaciones
/clientes	Clientes e historial
/financiero	Ingresos y egresos
/reportes	Consultas administrativas
/configuracion	Perfil, empresa, usuarios y sucursales
/consumo-empleados	Consumo interno habilitado
/cliente	Resumen mostrado al cliente
/llamado	Pedido listo y aviso por voz
/superadmin	Gestión global de empresas

Inicio y cierre de sesión

Ingresar a la URL suministrada por el proveedor y usar la cuenta individual. Confirmar nombre, empresa y sucursal. Si aparece una empresa equivocada, suspender operación y reportar. La sesión puede terminar por expiración del token o cambio de día; volver a ingresar. Salir mediante el control de cierre de sesión al dejar el puesto.

Preparación inicial del negocio

El administrador configura empresa, logo, sucursales y usuarios en Config. En Inventario registra insumos y unidades; en Productos crea categorías, productos, precios y recetas. Definir adicionales y verificar que el consumo de ingredientes corresponde a las unidades elegidas. Ejecutar una venta de ensayo antes de usar datos

reales.

No interpretar los planes como un listado contractual universal: documentar qué funciones quedaron habilitadas para la empresa. Consumo staff requiere habilitación específica. El interruptor de facturación electrónica no acredita una integración operativa.

Jornada del cajero

1. El supervisor abre caja en Dashboard: selecciona cajero e ingresa base inicial.
2. El cajero ingresa al POS con su cuenta asignada.
3. Selecciona productos, cantidades y modificaciones; revisa exclusiones y adicionales.
4. Asocia un cliente si procede y selecciona el método de pago realmente recibido.
5. Confirma una sola vez y comprueba número de pedido y total.
6. Entrega recibo según impresión habilitada y verifica comunicación a cocina.

Si aparece error después de confirmar, consultar si el pedido fue creado antes de repetir. El sistema registra el método de pago; no confirma una transferencia bancaria ni ejecuta un cobro de tarjeta.

Cocina y atención

Revisar el pedido completo, incluidos adicionales, exclusiones y observaciones. Actualizar su estado conforme a la operación. Marcar listo al finalizar y entregado cuando el cliente lo reciba. No cambiar a ANULADO para corregir una venta sin procedimiento del supervisor: actualmente no se revierten automáticamente los efectos asociados.

Las pantallas Cliente y Llamado deben abrirse en contextos compatibles del mismo navegador/origen que emite los avisos. Si se usan terminales independientes, requerir una solución de sincronización adicional. La voz depende de disponibilidad y permisos del navegador.

Inventario y preparaciones

ENTRADA suma al stock; SALIDA resta; AJUSTE establece un saldo. Anotar motivo y revisar historial. Al usar unidad de compra, confirmar factor: una conversión incorrecta afecta existencias y costos. El reporte de stock bajo compara con el mínimo configurado.

La pantalla incluye creación/listado de preparaciones; la API contiene registro de lotes. No se identificó formulario completo de captura de lotes en la UI revisada. Las porciones disponibles representan suma producida, no saldo real tras ventas; requieren revisión del responsable.

Clientes y fidelización

Buscar cliente antes de crear otro. Registrar solo datos necesarios para el servicio. Revisar historial y puntos. La empresa define cuánto comprar para ganar un punto y cuánto descuento representa al canjearlo. El programa inicia desactivado, conserva los saldos anteriores y permite excluir categorías y productos. La base elegible recibe su parte proporcional de los descuentos; domicilio excluido y redondeo hacia abajo. No hay valor obligatorio. El canje se realiza en POS y los ajustes manuales solo por administrador.

Finanzas y reportes

En Financiero registrar ingresos/egresos manuales con categoría y descripción; las ventas crean ingreso automáticamente, por lo que no deben duplicarse manualmente. Consultar períodos y reportes. El indicador de utilidad es ingresos menos egresos registrados, no una contabilidad integral. Diferencias entre reportes pueden requerir revisar anulaciones y filtros.

Consumo staff

Si está habilitado, administrador/gerente registra nombre del empleado, productos y cantidades. El consumo descuenta receta y no es venta. El valor mensual usa precio de referencia del producto; no es costo exacto de producción ni cobro al empleado.

Cierre de jornada

Conciliar efectivo y otros medios por separado; revisar ventas, eventos e incidencias. Ingresar monto final y cerrar caja en Dashboard. La aplicación compara contra base más ventas de todos los medios: acordar cómo se usa ese campo antes del cierre real. Si se omite monto, se puede usar esperado, lo que no acredita un conteo. Hay cierre automático fuera del horario codificado del servidor.

Soporte

Reportar pantalla, fecha/hora, usuario sin contraseña, sucursal, número de pedido, acción y mensaje de error. Adjuntar evidencia sin datos sensibles innecesarios. No compartir tokens, contraseñas ni respaldo completo por canales no acordados. Contacto, horario y tiempos de atención deben completarse en acta de entrega.

Mi tienda, Domicilios y Puntos

El administrador configura su página en Mi tienda y el programa en Puntos. Cajero, administrador y gerente revisan solicitudes en Domicilios; el domiciliario solo sus asignaciones. El comprador accede sin sesión del personal a /tienda/{slug}. Procedimientos completos en tiendas, domicilios y fidelización.

10. Plan de pruebas y matriz de trazabilidad

Estado de validación

La revisión 1.2 verifica documentación contra código y conserva evidencia de 15 escenarios HTTP con PostgreSQL aislado, dos pruebas unitarias existentes, compilación de API/web y comprobación de pantallas en navegador. No constituye aprobación productiva. Los 13 escenarios de servicios de 1.1 son evidencia histórica; sus brechas se reclasifican en el informe actualizado. Ver informe.

Se encontraron pruebas en `api/src/app.controller.spec.ts`, `api/src/impresion/impresion.service.spec.ts` y `api/test/app.e2e-spec.ts`. Cubren un saludo del controlador, impresión por socket simulado y ruta raíz e2e; no acreditan el ciclo de venta ni aislamiento multiempresa.

Preparación del ambiente

Crear ambiente aislado con dos empresas A/B, sucursales y cuentas de cada rol, catálogo mínimo y existencias conocidas. Usar únicamente datos sintéticos. Deshabilitar correo/Twilio reales y usar impresora simulada hasta prueba controlada de hardware. Controlar tareas cron y hora para que no cierren caja ni disparen mensajes durante las pruebas. Registrar commit y cambios locales incluidos, versiones y migraciones aplicadas.

Datos de cálculo sugeridos: producto a 10000, adicional a 2000, dos unidades con un adicional por unidad y descuento de 1000. Esperado: subtotal 24000, total 23000; los puntos dependen de la configuración explícita de prueba, no de una regla fija. Para stock, usar inicialmente ingredientes sin factor de conversión; probar conversiones en un caso separado.

Casos de prueba

La tabla es el plan completo. Los escenarios AT del informe cubren parcialmente CP-01, CP-05, CP-07, CP-08, CP-10 a CP-13, CP-19, CP-20, CP-22 y CP-25. No marcar un CP completo como aprobado cuando solo se probó un subcaso. Un resultado esperado puede representar una mejora requerida que hoy falle.

ID	Requisitos	Procedimiento resumido	Resultado esperado
CP-01	RF-01	Ingresar con contraseña correcta/incorrecta, usuario inactivo y empresa inactiva.	Solo acceso válido permitido; 401 controlado para rechazo.
CP-02	RF-02	Registrar empresa y repetir NIT/correo; inducir fallo al crear admin.	Rechazo de duplicados; documentar brecha de atomicidad ante fallo.
CP-03	RF-03, RF-04	Crear usuario/sucursal, editar perfil y desactivar; probar IDs de B con token A.	Operaciones permitidas limitadas a la empresa.
CP-04	RF-05, RF-06	Crear categoría padre/hija y producto con receta; desactivar producto e intentar vender.	Catálogo correcto y venta rechazada si no está disponible/activo.
CP-05	RF-07, RF-08	Vender datos de cálculo sugeridos; probar adicional ajeno.	23000 total y snapshot correcto; extra ajeno rechazado.

ID	Requisitos	Procedimiento resumido	Resultado esperado
CP-06	RF-08, RNF-04	Enviar items vacío, cantidad 0/negativa, descuento mayor al subtotal, ID inválido.	Rechazo sin persistencia parcial; registrar fallos actuales.
CP-07	RF-09	Vender sin caja, con caja ajena y como cajero asignado.	Rechazar los dos primeros y permitir el último.
CP-08	RF-10	Vender producto con un ingrediente excluido y extra con insumo.	Descuento exacto de receta no excluida y adicional.
CP-09	RF-11, RF-12	Abrir cocina, crear pedido, cambiar estado, cortar SSE.	Evento visible y recuperación por consulta periódica.
CP-10	RF-13, RF-14	Abrir caja, intentar segunda, vender por distintos medios y cerrar con diferencia >1000.	Una caja; cálculos actuales identificados y alerta registrada.
CP-11	RF-15	Registrar entrada, salida y ajuste con/sin conversión.	Saldo y movimiento coherentes, alerta bajo mínimo.
CP-12	RF-16, RF-17	Crear preparación/lote, consultar porciones y vender producto asociado.	Producción registrada; señalar que descuento de porciones aún no existe.
CP-13	RF-18	Asociar cliente a venta de 23000; sumar/redimir puntos y consultar historial.	Sin activar: cero puntos nuevos. Con reglas explícitas: cálculo elegible y canje de saldo correcto.
CP-14	RF-19, RF-20	Crear venta e ingreso/egreso manual; consultar periodos/reportes.	Totales conciliables con registros del período y empresa.
CP-15	RF-21	Simular socket; probar equipo real sin emitir venta; desconectar impresora.	Impresión confirmada solo cuando procede y fallo visible según configuración.
CP-16	RF-22	Abrir POS/cliente y cocina/llamado en mismo navegador; probar terminal separado.	Sincronización local; documentar ausencia de sincronización remota.
CP-17	RF-23	Usar proveedor de prueba; disparar alerta y cumpleaños con/sin configuración.	Evidencia de resultado sin enviar mensajes a personas reales.
CP-18	RF-24, RF-26	Usar SUPERADMIN y un usuario común para rutas globales; cambiar capacidad.	Solo rol autorizado; auditoría en operaciones instrumentadas.
CP-19	RF-25	Registrar consumo habilitado/deshabilitado y con rol cajero.	Solo supervisor habilitado; stock disminuye sin venta ni ingreso.
CP-20	RF-27	Anular venta y revisar inventario, puntos e ingreso.	Reversión integral pendiente; comprobar bloqueo de anulaciones web/con puntos y no reapertura.
CP-21	RF-28	Evaluar bandera y flujo completo de emisión.	No aceptar emisión como implementada hasta integración y prueba de proveedor.
CP-22	RNF-01, RNF-02	Probar lectura/escritura con IDs cruzados en clientes, inventario, caja y catálogo; invocar rutas con roles no autorizados.	Cero acceso indebido; fallos bloquean aceptación multiempresa.
CP-23	RNF-03	Inducir fallo entre pedido, stock, ingreso y puntos en ambiente aislado.	Todo o nada en venta; ampliar fallos inducidos en cada escritura.
CP-24	RNF-03, RNF-06	Crear ventas simultáneas en dos sucursales y ventas repetidas por reintento.	Numeración única, sin pérdida/duplicación; medir respuesta y colisiones.
CP-25	RNF-07	Respaldar DB/uploads y restaurar en destino aislado.	Datos consistentes y tiempo/pérdida medidos.

ID	Requisitos	Procedimiento resumido	Resultado esperado
CP-26	RNF-05, RNF-09	Revisar HTTPS, CORS, secretos, expiración y navegadores seleccionados.	Configuración segura y funciones soportadas documentadas.
CP-27	RNF-08	Usuario de prueba completa apertura, venta, cocina y cierre con teclado y táctil.	Flujo comprensible, errores visibles y sin duplicación.
CP-28	RNF-10, RNF-11, RNF-12	Compilar, revisar logs y simular dos instancias si se prevén.	Build reproducible, errores rastreables, eventos/tareas coordinados.

Matriz de trazabilidad por componente

Requerimientos	Componentes	Casos de uso	Pruebas
RF-01 a RF-04	auth, usuarios, sucursales	CU-01, CU-10	CP-01 a CP-03
RF-05 a RF-07	categorías, productos, adicionales	CU-02	CP-04, CP-05
RF-08 a RF-10	pedidos, caja	CU-03, CU-04	CP-05 a CP-08
RF-11, RF-12	pedidos, pedidos-eventos, cocina	CU-05	CP-09
RF-13, RF-14	caja, dashboard	CU-03, CU-08	CP-10
RF-15 a RF-17	inventario, preparaciones	CU-06, CU-07	CP-11, CP-12
RF-18	clientes, pedidos	CU-04	CP-13
RF-19, RF-20	financiero, reportes	CU-04, CU-08	CP-14
RF-21	impresión, pos	CU-04	CP-15
RF-22	cliente, llamado, cocina, dashboard	CU-05	CP-16
RF-23	notificaciones, tareas	CU-03 a CU-08	CP-17
RF-24, RF-26	superadmin, auditoría	CU-10	CP-18
RF-25	consumo-empleados	CU-09	CP-19
RF-27	pedidos, inventario, financiero, clientes	CU-04, CU-05	CP-20
RF-28	Pendiente; bandera en empresa	CU-10	CP-21
RNF-01 a RNF-12	Transversales	Todos	CP-06, CP-22 a CP-28

Criterios de entrada y salida

Entrada: ambiente aislado, datos restaurables, configuración registrada y responsables identificados. Salida propuesta: todos los casos críticos de venta, autorización, aislamiento, inventario, caja y recuperación aprobados; defectos menores con plan y aceptación explícita; ninguna función pendiente presentada como entregada. Rendimiento y soporte necesitan metas acordadas antes de aceptación.

Registro de ejecución por completar

Campo	Valor
Caso / fecha / ejecutor	Pendiente
Versión y ambiente	Pendiente
Datos utilizados	Pendiente
Resultado real	Pendiente
Evidencia	Pendiente
Estado	Pendiente / aprobado / fallido / bloqueado
Incidencia y responsable	Pendiente
Reprueba y aprobación	Pendiente

Trazabilidad añadida en 1.2

Caso	Requisitos	Caso de uso	Verificación
CP-29	RF-29, RF-30	CU-11, CU-12	Tienda automática, catálogo aislado, edición autorizada y precio actualizado.
CP-30	RF-31, RF-36	CU-12	Solicitud pública, ID ajeno, cantidad inválida, total manipulado, reintento idempotente y seguimiento.
CP-31	RF-32, RNF-03	CU-13	Caja cerrada sin venta; dos aceptaciones concurrentes crean una sola venta.
CP-32	RF-33	CU-13	Asignación a repartidor propio y entrega sincronizada; prohibir acceso de otra empresa.
CP-33	RF-34	CU-15	WhatsApp correcto y texto del carrito; no afirmar envío/importación automática.
CP-34	RF-35, RF-18	CU-14	Regla desactivada inicial, exclusiones, canje proporcional, saldo insuficiente y canjes concurrentes.

El script `api/test/tienda-integracion.cjs` verifica 15 escenarios concretos de estos casos, no todas sus combinaciones. Requiere `TIENDA_TEST_DATABASE_URL` apuntando a la base aislada `powerpos_store_test` en `127.0.0.1:55439`. Los ensayos de navegador comprobaron carrito, seguimiento tras recarga, bandeja, configuración y ancho móvil; no sustituyen evaluación de accesibilidad o carga exhaustiva.

11. Brechas, riesgos y evolución

Hallazgos de lectura del código, no una auditoría exhaustiva ni una prueba de explotación. Prioridad propuesta: **P0** bloquea aceptación del alcance afectado; **P1** resolver antes de operación estable; **P2** mejora planificada. La decisión de entrega corresponde a responsables del proyecto y cliente con evidencia de pruebas.

Pendientes para el despliegue

ID	Prioridad	Evidencia y efecto	Acción y verificación
B-01	P0 multiempresarial	Ingrediente sin empresa/sucursal e InventarioService sin filtro empresarial. Puede compartirse stock entre negocios.	Definir propietario y stock por ámbito, migrar datos y aprobar CP-22.
B-02	P0 parcial	Cientes ahora filtra por empresa; se verificó rechazo de cliente ajeno. Cierre de caja por ID sigue requiriendo revisión.	Completar matriz de caja y demás recursos.
B-03	P0 acceso	Numerosos controladores solo verifican JWT. Caja no declara autorización de supervisor aunque UI lo indica.	Matriz de permisos de servidor y pruebas por cada rol.
B-04	P0 parcial	Venta y aceptación web ahora son transaccionales; consumo y lotes conservan revisión pendiente.	Ampliar fallos inducidos y transacciones de módulos restantes.
B-05	P0 si se ofrece devolución	Las solicitudes web RECIBIDO se pueden rechazar sin venta. Una venta web aceptada o con movimientos de puntos bloquea la anulación directa y requiere conciliación. No existe todavía un flujo automático de devolución que revierta ingreso e inventario; no presentar ANULADO como reembolso.	Diseñar compensación idempotente.
B-06	P1 parcial	Cantidades, descuentos, canjes y entradas de tienda tienen validaciones; no hay ValidationPipe global.	Completar validación uniforme de otros módulos.
B-07	Mitigado / P1 reintentos POS	Número usa sucursal y UUID; solicitudes web y su aceptación tienen controles de duplicados. POS directo no tiene clave de idempotencia.	Añadir garantía de reintento a venta POS.
B-08	P1	Apertura de caja usa consulta previa sin restricción única condicional observada; peticiones simultáneas pueden competir.	Garantía de concurrencia y prueba paralela de apertura.
B-09	P1	Cierre suma todos los pagos a base; monto omitido usa esperado. Cron cierra fuera de horario fijo.	Definir arqueo por medio, conteo real, horarios y zona por sucursal.
B-10	P1	Stock puede quedar negativo; ventas y consumo descuentan directamente sin MovimientoInventario.	Política de disponibilidad y registro de todos los movimientos.

ID	Prioridad	Evidencia y efecto	Acción y verificación
B-11	P1 si se ofrece saldo de preparaciones	Porciones disponibles solo suman lotes; venta no consume ProductoPreparacion.	Diseñar consumo/asignación de porciones y evitar doble descuento de insumos.
B-12	P1 parcial	HTTP principal/tienda usan NEXT_PUBLIC_API_URL; quedan referencias locales de SSE/impresión, CORS abierto y secreto JWT de respaldo.	Completar configuración por ambiente.
B-13	P1	JWT conserva actividad/rol previos hasta expiración; token query puede aparecer en logs.	Definir revocación/revalidación y manejo de logs; CP-26.
B-14	P1	Creación pública de empresa y admin no es transacción integral; acceso público sin limitación observada.	Acordar aprovisionamiento público o administrado y reforzar alta.
B-15	P1	Fallback sucursal 1 y alertas de inventario empresa 1; algunos avisos no reciben empresald.	Eliminar supuestos de tenant fijo y validar destinatarios.
B-16	P1 parcial	Restauración sintética verificada en 1.1 y backup local previo a migración 1.2; sin garantía de copias futuras o recuperación de uploads.	Automatizar copias externas y probar recuperación integral.
B-17	P2 o P1 al escalar	SSE en memoria, cron por proceso, Redis sin integración observada.	Coordinar eventos y tareas antes de varias instancias.
B-18	P1 alcance comercial	Bandera de factura no implementa emisión; planes/permisos no tienen enforcement global acreditado.	Delimitar oferta y completar integración solo si se contrata.
B-19	P2	Pantallas cliente/llamado usan comunicación local del navegador.	Documentar requisito del puesto o implementar canal remoto.
B-20	P1 aceptación	15 escenarios HTTP de tienda/puntos y builds API/web aprobados; faltan validación operativa y cobertura completa.	Ejecutar piloto, hardware, carga, aislamiento restante y recuperación.

Orden de trabajo propuesto

1. **Alcance y consistencia:** decidir si entrega es de una empresa o varias, matriz de permisos, inventario, caja y anulación; corregir B-01 a B-10 según alcance.
2. **Implantación:** configuración por ambiente, credenciales, red de impresión, persistencia, respaldo y hora; completar pruebas de restauración.
3. **Aceptación:** ejecutar casos con datos de ensayo, registrar defectos y capacitar personal.
4. **Entrega:** registrar versión, capacidades efectivas, limitaciones, soporte y firmas.
5. **Evolución:** preparación con saldo real, factura electrónica si se contrata, escalado y sincronización remota.

No se asignan fechas ni esfuerzo porque no se proporcionó capacidad del equipo ni fecha comprometida. La revisión distingue correcciones implementadas en 1.2 de pendientes del sistema.

Criterio de mantenimiento documental

Cambios de Prisma: regenerar modelo/diccionario y revisar migraciones. Cambios de rutas: regenerar catálogo y contratos. Cambios de negocio: actualizar requerimiento, caso de uso, manual y prueba vinculada. Cada versión entregada debe identificar commit o paquete exacto; esta versión documental incluye cambios locales aún no consolidados.

Aprobación académica y comercial

Para la entrega académica, explicar la diferencia entre objetivos, capacidades observadas y resultados probados. Para compradores, entregar el manual operativo y acta con funciones habilitadas y limitaciones aceptadas. Mantener este registro técnico disponible para mantenimiento y decisiones de despliegue, sin convertir objetivos propuestos en garantías contractuales.

Límites del nuevo canal público

No hay pasarela de pago, reserva de inventario al solicitar, geocodificación ni integración automática de conversaciones WhatsApp. Límites iniciales por IP en memoria y teléfono persistente requieren refuerzo distribuido al escalar. No exponer saldos de puntos por teléfono; el personal verifica identidad al asociar cliente.

Tienda por empresa, domicilios y fidelización configurable

Versión funcional 1.2 · 17 de septiembre de 2026. Integrado en los manuales y entregables 1.2. Sustituye las reglas fijas de puntos de versiones anteriores.

Qué se incorpora

Cada empresa recibe automáticamente un identificador de tienda al crearse. Su página se sirve en `/tienda/{tiendaSlug}` dentro del mismo sitio de PowerPOS. No se crea un despliegue independiente por empresa. La migración asigna identificadores también a las empresas existentes.

La tienda muestra nombre y logo de la empresa, título, descripción, portada opcional, color, horario, catálogo, precios, disponibilidad y contacto. La vista inicial funciona para restaurantes, bares y comercios; no exige funciones de cocina para comprar.

Administrador: publicar y alimentar la tienda

1. Entrar a **Mi tienda**. Copiar el enlace o personalizar su identificador con letras minúsculas, números y guiones. Cambiarlo invalida enlaces anteriores.
2. En **Configuración**, guardar nombre, logo, dirección y teléfono de la empresa.
3. En **Mi tienda**, escribir título, descripción y horario; elegir color y opcionalmente una portada HTTPS.
4. Registrar el WhatsApp de esa empresa con indicativo de país, solo números.
5. Administrar categorías, productos, imágenes y precios desde **Productos**. La tienda consulta ese mismo catálogo, sin copiar precios a otra base.
6. Seleccionar sucursal receptora, definir compra mínima y zonas con tarifas.
7. Activar **Recibir pedidos desde la web**. Por defecto la página existe, pero la recepción de pedidos permanece apagada hasta configurar su operación.

Las modificaciones las autoriza el servidor según empresa y rol. Solo ADMIN_EMPRESA configura tienda y fidelización; el catálogo mantiene administración por administrador/gerente. Los precios públicos se consultan al cargar la página. Al enviar y aceptar se vuelven a comprobar en el servidor.

Comprador: realizar un pedido

1. Abrir la tienda, buscar o filtrar productos y agregarlos al carrito.
2. Indicar nombre, teléfono, dirección, zona, forma de pago e instrucciones.
3. Revisar productos, tarifa y total, y autorizar el uso de esos datos para gestionar el domicilio.
4. Enviar. El servidor obtiene los precios de su catálogo; no utiliza precios enviados por el navegador.
5. Recibir una referencia y un enlace con `?pedido=...` que permite recuperar el seguimiento al recargar. No compartir ese enlace innecesariamente.

La solicitud no acredita un pago ni crea una venta hasta que el cajero la acepte. No se incorporó una pasarela de pagos. EFECTIVO, TRANSFERENCIA, NEQUI y DAVIPLATA son medios declarados que el negocio debe verificar.

WhatsApp: abre una conversación con los productos y el subtotal propuestos. No envía mensajes automáticamente ni sincroniza conversaciones con PowerPOS. El personal registra y confirma esos pedidos por su procedimiento habitual; una integración automática con WhatsApp Business requeriría credenciales y configuración adicionales.

Cajero y domiciliario

La navegación del cajero muestra el número de solicitudes pendientes, consultado cada cinco segundos.

Domicilios permite revisar datos, productos, total e instrucciones. El cajero ve solo su sucursal; administrador/gerente pueden revisar su empresa.

Estado	Acción y efecto
RECIBIDO	Solicitud pendiente. No afecta inventario, caja, ingresos ni puntos.
RECHAZADO	El cajero indica un motivo. No crea una venta.
ACEPTADO	El cajero verifica el pedido y el medio de pago. Con caja abierta, se crea una sola venta, sus detalles, inventario, ingreso y puntos en una transacción.
EN_CAMINO	Se asigna un usuario DOMICILIARIO activo de la misma empresa y se despacha.
ENTREGADO	El domiciliario asignado o el personal autorizado confirma la entrega y se actualiza también el pedido del POS.

El domiciliario inicia sesión en **Domicilios** y solo ve sus asignaciones. Dos aceptaciones simultáneas no producen dos ventas. Un fallo de la aceptación revierte todos sus cambios. Si el catálogo cambió respecto de la solicitud, se debe contactar al comprador antes de tramitar una nueva solicitud con el precio correcto.

La cocina puede avanzar la preparación, pero la entrega de pedidos web se confirma desde Domicilios. Rechazar una solicitud pendiente no equivale a anular una venta aceptada. Las cancelaciones posteriores a aceptación y ventas con movimientos de puntos requieren conciliación; no se implementó un flujo automático de devoluciones/reembolsos.

Fidelización: dos valores definidos por cada empresa

Por decisión del responsable del proyecto, no existe una tarifa obligatoria ni se conserva como regla activa «1 punto por cada \$1.000». El programa comienza **desactivado**, con ambos valores sin definir, y conserva los saldos históricos existentes.

En **Puntos**, el ADMIN_EMPRESA define:

- Compra elegible necesaria para ganar un punto, en COP.
- Descuento en COP que representa un punto al canjearlo.
- Categorías que no generan puntos.
- Productos que no generan puntos.

Las exclusiones se suman. Las subcategorías se seleccionan individualmente; excluir una categoría no excluye automáticamente sus hijas. Los adicionales siguen la elegibilidad de su producto principal. Los descuentos se distribuyen proporcionalmente entre las líneas. El domicilio no genera puntos ni puede pagarse con ellos. Se redondea la acumulación hacia abajo al punto entero.

Ejemplo exclusivamente de prueba, no tarifa recomendada: reglas de \$1.000 por punto y \$100 de descuento por punto. Una venta de comida por \$20.000 y bebidas excluidas por \$10.000 acumula 20 puntos. Canjear 10 puntos descuenta \$1.000; la porción de descuento correspondiente a la comida reduce la base elegible y la venta acumula 19 puntos.

En POS, seleccionar un cliente identificado, indicar puntos a canjear y revisar el descuento. El backend verifica el saldo, registra puntos ganados/canjeados y valor aplicado en el pedido, y actualiza el saldo en la misma transacción. El canje aislado anterior en Clientes se reemplaza por canje dentro de la venta. Los ajustes positivos manuales quedan reservados al administrador.

Los compradores públicos no pueden reclamar un saldo por indicar un teléfono. Para acumular puntos en pedidos web, el cajero verifica su identidad y asocia el cliente correcto al aceptar. No se habilita consulta ni canje público de saldos sin autenticación del cliente.

Cambiar el valor monetario afecta los canjes futuros del saldo existente; no cambia los descuentos ni puntos guardados en ventas anteriores.

Activación técnica

1. Respaldar la base real antes de migrar.
2. En `api`, ejecutar `npx prisma migrate deploy` y `npx prisma generate`, luego compilar y reiniciar la API.
3. Antes de compilar `web`, definir `NEXT_PUBLIC_API_URL` con la URL HTTPS pública de la API. Si se omite, se conserva el destino local `http://localhost:3000` para desarrollo.
4. Compilar/reiniciar la aplicación web. La tienda vive en ese despliegue, no en la landing comercial de Sites.
5. Verificar las URLs de logos e imágenes. Un enlace `localhost` no será accesible para compradores remotos.
6. Configurar sucursal, zonas, WhatsApp, recepción de pedidos y reglas de fidelización desde el panel.

La migración `20260916070000_tienda_domicilios_fidelizacion` se probó primero en PostgreSQL aislado y luego se aplicó satisfactoriamente en la base local `powerpos_dev` el 17 de septiembre de 2026. Antes se creó y se comprobó el catálogo del respaldo `backups/powerpos_pre_tienda_20260917.dump` (fuera de Git). No se desplegó en un entorno público. En otras instalaciones deben seguirse los pasos anteriores; en esta base local no es necesario repetir la migración.

Validación y límites

Prueba reproducible: `api/test/tienda-integracion.cjs`. Exige `TIENDA_TEST_DATABASE_URL` apuntando exactamente a `127.0.0.1:55439/powerpos_store_test`; rechaza otro destino. Requiere migraciones y backend compilado. Crea empresas y usuarios sintéticos; no borra información.

Resultado: 15 escenarios HTTP satisfactorios con PostgreSQL aislado: catálogo aislado, roles, manipulación de precios, reintentos, caja cerrada, aceptación concurrente, entrega, exclusiones de puntos, canje proporcional, saldo concurrente y rechazo sin venta. Las dos pruebas existentes del backend también pasan. API y aplicación

web compilan con sus nuevas rutas. En navegador se verificó envío desde el carrito, recuperación del seguimiento tras recargar, aparición en la bandeja, pantallas de configuración y vista móvil sin desbordamientos ni errores JavaScript.

La protección inicial contra abuso usa un límite en memoria por IP y uno persistente por teléfono/empresa. Para varios servidores debe añadirse un límite distribuido en el despliegue. Las zonas son seleccionadas por el cliente y revisadas por el cajero; no hay geocodificación ni cálculo automático de distancias. No se reserva inventario durante la solicitud pendiente. La tienda utiliza cantidades enteras; no incorpora integración de básculas ni lectores como parte de esta entrega.

Las brechas históricas de inventario global y conciliaciones descritas en la documentación 1.1 no se consideran corregidas por este módulo. La activación productiva requiere revisar esos pendientes y ejecutar el piloto.

Catálogo de rutas declarado en código

Generado desde controladores NestJS. Incluye decoradores, parámetros anotados y referencias; no es OpenAPI ni garantiza validación de cuerpos. `any` requiere consultar servicio. Los roles reflejan decoradores, no reglas inferidas de la interfaz.

/adicionales

Fuente: `api/src/adicionales/adicionales.controller.ts`. Clase: `AdicionalesController`. Guards de clase: `JwtAuthGuard`.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET	<code>/adicionales</code>	Sin restricción de rol declarada	<code>Query(productoId)</code>	—
POST	<code>/adicionales</code>	Sin restricción de rol declarada	<code>Body()</code>	—
PATCH	<code>/adicionales/:id</code>	Sin restricción de rol declarada	<code>Param(id); Body()</code>	—
DELETE	<code>/adicionales/:id</code>	Sin restricción de rol declarada	<code>Param(id)</code>	—

/

Fuente: `api/src/app.controller.ts`. Clase: `AppController`. Guards de clase: ninguno declarado.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET	<code>/</code>	Sin restricción de rol declarada	—	—

/auth

Fuente: `api/src/auth/auth.controller.ts`. Clase: `AuthController`. Guards de clase: ninguno declarado.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
POST	<code>/auth/login</code>	Sin restricción de rol declarada	<code>Body()</code>	—
POST	<code>/auth/registrar-empresa</code>	Sin restricción de rol declarada	<code>Body()</code>	—

/caja

Fuente: `api/src/caja/caja.controller.ts`. Clase: `CajaController`. Guards de clase: `JwtAuthGuard`.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
POST	/caja/abrir	Sin restricción de rol declarada	Body()	—
POST	/caja/:id/cerrar	Sin restricción de rol declarada	Param(id); Body()	—
GET	/caja/abierta	Sin restricción de rol declarada	—	—
GET	/caja/eventos	Sin restricción de rol declarada	—	—
GET	/caja/alertas	Sin restricción de rol declarada	—	—
POST	/caja/apertura-irregular	Sin restricción de rol declarada	Body()	—

/categorias

Fuente: api/src/categorias/categorias.controller.ts. Clase: CategoriasController. Guards de clase: JwtAuthGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
POST	/categorias	Sin restricción de rol declarada	Body()	—
GET	/categorias	Sin restricción de rol declarada	—	—
PATCH	/categorias/:id	Sin restricción de rol declarada	Param(id); Body()	—
DELETE	/categorias/:id	Sin restricción de rol declarada	Param(id)	—

/clientes

Fuente: api/src/clientes/clientes.controller.ts. Clase: ClientesController. Guards de clase: JwtAuthGuard, RolesGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET	/clientes	ADMIN_EMPRESA, GERENTE, CAJERO	Query(busqueda)	—
POST	/clientes	ADMIN_EMPRESA, GERENTE, CAJERO	Body()	—
GET	/clientes/:id	ADMIN_EMPRESA, GERENTE, CAJERO	Param(id)	—
PATCH	/clientes/:id	ADMIN_EMPRESA, GERENTE, CAJERO	Param(id); Body()	—
PATCH	/clientes/:id/toggle-activo	ADMIN_EMPRESA, GERENTE, CAJERO	Param(id)	—
POST	/clientes/:id/puntos/agregar	ADMIN_EMPRESA	Param(id); Body(puntos)	—
POST	/clientes/:id/puntos/redimir	ADMIN_EMPRESA, GERENTE, CAJERO	Param(id); Body(puntos)	—

/consumo-empleados

Fuente: api/src/consumo-empleados/consumo-empleados.controller.ts. Clase: ConsumoEmpleadosController. Guards de clase: JwtAuthGuard, RolesGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
POST	/consumo-empleados	ADMIN_EMPRESA, GERENTE	Body()	—
GET	/consumo-empleados	ADMIN_EMPRESA, GERENTE	Query(sucursalId)	—
GET	/consumo-empleados/resumen	ADMIN_EMPRESA, GERENTE	—	—

/empresa

Fuente: api/src/empresa/empresa.controller.ts. Clase: EmpresaController. Guards de clase: JwtAuthGuard, RolesGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET	/empresa	Sin restricción de rol declarada	—	—
PATCH	/empresa	ADMIN_EMPRESA	Body()	—
POST	/empresa/logo	ADMIN_EMPRESA	UploadedFile()	—

/financiero

Fuente: api/src/financiero/financiero.controller.ts. Clase: FinancieroController. Guards de clase: JwtAuthGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
POST	/financiero/movimiento	Sin restricción de rol declarada	Body()	—
GET	/financiero/movimientos	Sin restricción de rol declarada	Query()	—
GET	/financiero/resumen	Sin restricción de rol declarada	Query(fechaDesde); Query(fechaHasta)	—

/impresion

Fuente: api/src/impresion/impresion.controller.ts. Clase: ImpresionController. Guards de clase: JwtAuthGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
POST	/impresion/comanda	Sin restricción de rol declarada	Body()	—
POST	/impresion/ticket	Sin restricción de rol declarada	Body()	—
POST	/impresion/abrir-cajon	Sin restricción de rol declarada	—	—

/inventario

Fuente: api/src/inventario/inventario.controller.ts. Clase: InventarioController. Guards de clase: JwtAuthGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET	/inventario	Sin restricción de rol declarada	Query(incluirInactivos)	—
POST	/inventario	Sin restricción de rol declarada	Body()	—
GET	/inventario/alertas	Sin restricción de rol declarada	—	—
GET	/inventario/:id/historial	Sin restricción de rol declarada	Param(id)	—
POST	/inventario/:id/ajuste	Sin restricción de rol declarada	Param(id); Body()	—
PATCH	/inventario/:id	Sin restricción de rol declarada	Param(id); Body()	—
PATCH	/inventario/:id/toggle-activo	Sin restricción de rol declarada	Param(id)	—

/pedidos

Fuente: `api/src/pedidos/pedidos.controller.ts`. Clase: `PedidosController`. Guards de clase: `JwtAuthGuard`, `RolesGuard`.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET (SSE)	/pedidos/stream	Sin restricción de rol declarada	—	—
POST	/pedidos	CAJERO, ADMIN_EMPRESA, GERENTE	Body()	—
GET	/pedidos	CAJERO, COCINERO, ADMIN_EMPRESA, GERENTE, DOMICILIARIO	Query(sucursalId)	—
GET	/pedidos/estadisticas	ADMIN_EMPRESA, GERENTE	—	—
GET	/pedidos/:id	CAJERO, COCINERO, ADMIN_EMPRESA, GERENTE, DOMICILIARIO	Param(id)	—
PATCH	/pedidos/:id/estado	COCINERO, CAJERO, ADMIN_EMPRESA, GERENTE, DOMICILIARIO	Param(id); Body()	—

/preparaciones

Fuente: `api/src/preparaciones/preparaciones.controller.ts`. Clase: `PreparacionesController`. Guards de clase: `JwtAuthGuard`.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET	/preparaciones	Sin restricción de rol declarada	—	—
GET	/preparaciones/:id	Sin restricción de rol declarada	Param(id)	—
POST	/preparaciones	Sin restricción de rol declarada	Body()	—
PATCH	/preparaciones/:id	Sin restricción de rol declarada	Param(id); Body()	—

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
DELETE	/preparaciones/:id	Sin restricción de rol declarada	Param(id)	—
GET	/preparaciones/:id/lotos	Sin restricción de rol declarada	Param(id)	—
POST	/preparaciones/:id/lotos	Sin restricción de rol declarada	Param(id); Body()	—

/productos

Fuente: api/src/productos/productos.controller.ts. Clase: ProductosController. Guards de clase: JwtAuthGuard, RolesGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
POST	/productos	ADMIN_EMPRESA, GERENTE	Body()	—
GET	/productos	Sin restricción de rol declarada	Query(categoriald)	—
GET	/productos/:id	Sin restricción de rol declarada	Param(id)	—
PATCH	/productos/:id	ADMIN_EMPRESA, GERENTE	Param(id); Body()	—
DELETE	/productos/:id	ADMIN_EMPRESA, GERENTE	Param(id)	—

/reportes

Fuente: api/src/reportes/reportes.controller.ts. Clase: ReportesController. Guards de clase: JwtAuthGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET	/reportes/empleados	Sin restricción de rol declarada	—	—
GET	/reportes/clientes	Sin restricción de rol declarada	Query(limite)	—
GET	/reportes/periodos	Sin restricción de rol declarada	—	—
GET	/reportes/rentabilidad	Sin restricción de rol declarada	—	—
GET	/reportes/periodo	Sin restricción de rol declarada	Query(tipo); Query(fecha)	—
GET	/reportes/diario	Sin restricción de rol declarada	Query(fecha)	—
GET	/reportes/mensual	Sin restricción de rol declarada	Query(fecha)	—
GET	/reportes/anual	Sin restricción de rol declarada	Query(fecha)	—

/sucursales

Fuente: api/src/sucursales/sucursales.controller.ts. Clase: SucursalesController. Guards de clase: JwtAuthGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
POST	/sucursales	Sin restricción de rol declarada	Body()	—
GET	/sucursales	Sin restricción de rol declarada	Query(incluirInactivas)	—
PATCH	/sucursales/:id	Sin restricción de rol declarada	Param(id); Body()	—
PATCH	/sucursales/:id/toggle-activo	Sin restricción de rol declarada	Param(id)	—

/superadmin

Fuente: api/src/superadmin/superadmin.controller.ts. Clase: SuperAdminController. Guards de clase: JwtAuthGuard, RolesGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET	/superadmin/resumen	SUPERADMIN	—	—
GET	/superadmin/empresas	SUPERADMIN	—	—
GET	/superadmin/auditoria	SUPERADMIN	—	—
POST	/superadmin/empresas	SUPERADMIN	Body()	—
PATCH	/superadmin/empresas/:id/estado	SUPERADMIN	Param(id); Body()	—
PATCH	/superadmin/empresas/:id/configuracion	SUPERADMIN	Param(id); Body()	—

/tareas

Fuente: api/src/tareas/tareas.controller.ts. Clase: TareasController. Guards de clase: JwtAuthGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
POST	/tareas/probar-cumpleaños	Sin restricción de rol declarada	—	—

/tiendas

Fuente: api/src/tienda/tienda.controller.ts. Clase: TiendaPublicaController. Guards de clase: ninguno declarado.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET	/tiendas/:slug	Sin restricción de rol declarada	Param(slug)	—
POST	/tiendas/:slug/pedidos	Sin restricción de rol declarada	Param(slug); Body()	PedidosPublicosGuard
GET	/tiendas/:slug/pedidos/:id	Sin restricción de rol declarada	Param(slug); Param(id)	—

/tienda-admin

Fuente: api/src/tienda/tienda.controller.ts. Clase: TiendaAdminController. Guards de clase: JwtAuthGuard, RolesGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
GET	/tienda-admin/resumen	ADMIN_EMPRESA, GERENTE, CAJERO	—	—
GET	/tienda-admin/configuracion	ADMIN_EMPRESA, GERENTE, CAJERO	—	—
PATCH	/tienda-admin/configuracion	ADMIN_EMPRESA	Body()	—
GET	/tienda-admin/pedidos	ADMIN_EMPRESA, GERENTE, CAJERO, DOMICILIARIO	—	—
GET	/tienda-admin/repartidores	ADMIN_EMPRESA, GERENTE, CAJERO	—	—
PATCH	/tienda-admin/pedidos/:id	ADMIN_EMPRESA, GERENTE, CAJERO, DOMICILIARIO	Param(id); Body()	—

/usuarios

Fuente: api/src/usuarios/usuarios.controller.ts. Clase: UsuariosController. Guards de clase: JwtAuthGuard.

Método	Ruta	Roles declarados	Entradas anotadas	Guards de método
POST	/usuarios	Sin restricción de rol declarada	Body()	—
GET	/usuarios	Sin restricción de rol declarada	—	—
GET	/usuarios/mi-perfil	Sin restricción de rol declarada	—	—
PATCH	/usuarios/mi-perfil	Sin restricción de rol declarada	Body()	—
GET	/usuarios/:id	Sin restricción de rol declarada	Param(id)	—
PATCH	/usuarios/:id	Sin restricción de rol declarada	Param(id); Body()	—
PATCH	/usuarios/:id/permisos	Sin restricción de rol declarada	Param(id); Body(permisos)	—
PATCH	/usuarios/:id/toggle-activo	Sin restricción de rol declarada	Param(id)	—